

Analisi dell'architettura Temporal VAE per la stima del Value-at-Risk

Mattia Corigliano Mat. 264444

18 gennaio 2026

Sommario

La stima del Value-at-Risk (VaR) richiede la modellazione accurata della coda sinistra della distribuzione dei rendimenti, in presenza di eteroschedasticità, non-linearità e possibili cambi di regime che rendono spesso insufficienti ipotesi parametriche semplici e modelli econometrici classici. In questo lavoro si analizza e si ricostruisce l'architettura *Temporal Variational Autoencoder* (TempVAE) proposta da Sick et al. (2023), un modello generativo per serie temporali che introduce variabili latenti dinamiche e utilizza GRU per catturare dipendenze temporali, stimando in modo flessibile media e covarianza (con parametrizzazione low-rank) dei rendimenti multi-asset. A causa dell'assenza di una implementazione pubblica nel paper originale, il modello è stato replicato via *reverse engineering* e validato tramite backtesting del VaR al 95% (test di Kupiec e di Christoffersen). Nella prima fase (2018–2021) la replica risulta statisticamente calibrata. Estendendo l'analisi a un campione longitudinale più ampio (2004–2025) emergono limiti della configurazione baseline, in particolare clustering delle violazioni e fallimento della copertura condizionale in fasi di stress. Per migliorare la reattività e l'aderenza alle dinamiche di volatilità, si propone una versione *tuned* basata su modifiche mirate degli iperparametri. I risultati mostrano una maggiore fedeltà distributiva (riduzione della NLL) e uno spazio latente più informativo (KL più alta, minore rischio di *posterior collapse*), mantenendo la validità del VaR e migliorando la robustezza nei regimi di crisi.

1 Introduzione

L'argomento cardine dell'analisi è il **VaR**, acronimo di **Value At Risk**, ovvero un indicatore di rischio che, relativamente a un investimento, misura la perdita che non sarà superata con un determinato livello di confidenza in un certo orizzonte temporale. Si tratta di un indicatore di rischio utilizzabile nelle decisioni finanziarie, ma non solo. Il VaR, risponde a una singola domanda: "Qual è la peggiore perdita che potrei subire nel prossimo periodo (es. domani), escludendo gli scenari estremi o catastrofici?" Se diciamo che un portafoglio ha un VaR giornaliero di 1 milione di euro al 95% di confidenza, stiamo dicendo che:

- Nel 95% dei giorni, le perdite saranno inferiori a 1 milione.
- C'è solo il 5% di probabilità (circa un giorno su 20) che la perdita sia superiore a 1 milione.

Dal punto di vista statistico, se definiamo L come la variabile aleatoria della perdita (Loss), il VaR è semplicemente il quantile $(1 - \alpha)$ della distribuzione delle perdite:

$$P(L > \text{VaR}_\alpha) \leq 1 - \alpha$$

Dove α indica un certo livello di confidenza, solitamente pari al 95% o 99%. Questo valore viene spesso usato presupponendo l'ipotesi di normalità dei rendimenti, secondo la quale le perdite e i ricavi dell'investimento si distribuiscono secondo una normale con media pari al rendimento medio e varianza pari alla volatilità dell'investimento. Per il calcolo della varianza di un portafoglio è necessario conoscere le correlazioni reciproche fra i titoli facenti parte del portafoglio, facendo ricorso alla matrice di varianze e covarianze. Lo studio del VaR di un portfolio non serve per anticipare cambiamenti nella composizione del portafoglio durante la giornata. Ma semplicemente ha il compito di riflettere il rischio del portafoglio in modo informativo, data l'attuale composizione del portafoglio.

L'approccio più comune è stato sviluppato nell'ambito dell'econometria finanziaria, dove si assume tipicamente che i prezzi degli strumenti finanziari formino processi stocastici condizionatamente eteroschedastici, la cui specificazione esatta, insieme alla loro stima e inferenza statistica, costituisce una parte fondamentale degli sforzi dei ricercatori. Un gruppo fondamentale di questo tipo di strumenti per la modellazione e la previsione della volatilità (e, di conseguenza, del rischio) sono i modelli **GARCH** sviluppati da Bollerslev e Taylor (1986), che generalizzano la specificazione **ARCH** proposta da Engle (1982). Le informazioni di input nei modelli GARCH, che determinano la volatilità attuale, comprendono principalmente i tassi di rendimento passati e le loro varianze condizionate. Numerose ricerche successive hanno mirato a modifiche ed estensioni della struttura GARCH originale, anche ammettendo vari tipi di distribuzione condizionale. Ciò ha portato a una notevole diversità della classe GARCH, con alcune delle specificazioni più diffuse. Una tendenza parallela nella modellazione e previsione delle serie temporali finanziarie segue lo sviluppo di strumenti di apprendimento automatico, in particolare delle reti neurali artificiali (ANN). Questi modelli, spesso trattati come "scatole nere", sono considerati tecniche non lineari e non parametriche in cui non viene formulata alcuna ipotesi a priori riguardante la forma matematica del modello. La funzione di mappatura dei dati di input in segnali di output (previsioni) viene realizzata nella fase di training del modello, implementato sulla base di un learning set comprendente quotazioni storiche. Questi strumenti includono principalmente reti neurali ricorrenti basate sull'apprendimento profondo (RNN; introdotte da Hopfield, 1982), in particolare reti di memoria a lungo e breve termine (LSTM; Hochreiter e Schmidhuber, 1997), e anche reti neurali Gated Recurrent Unit (GRU) (Chung et al., 2014), che sono alla base dell'architettura presentata nel paper in analisi. La transizione al Deep Learning avviene per avere modelli più robusti, flessibili e capaci di 'vedere' pattern di rischio che sfuggono alle equazioni tradizionali. I motivi principali è per superare i limiti strutturali dei modelli econometrici classici come il GARCH:

- attraverso reti più complesse è più facile **catturare le non-linearità**, dove i modelli classici faticano.

- Siccome **dati finanziari** sono **molto rumorosi**, allora architetture come i VAE sono progettate per imparare una rappresentazione latente "compressa" e robusta dei dati, filtrando il rumore e concentrandosi sui fattori di rischio sottostanti (il "segnale").
- **Adattabilità Temporale:** dove le reti ricorrenti (RNN, LSTM, GRU) sono eccellenti nel catturare la dipendenza temporale a lungo termine (es. la volatilità di oggi dipende da shock avvenuti molto tempo fa), cosa che i modelli autoregressivi classici faticano a fare in modo efficiente su lunghe finestre.

L'indagine si focalizza sul contributo di Sick et al. (2023) dal titolo "*Estimating the Value-at-Risk by Temporal VAE*". L'obiettivo primario risiede nella decostruzione analitica, nell'interpretazione formale e nella validazione empirica dell'architettura proposta, denominata **Temporal Variational Autoencoder (TempVAE)**. Data l'assenza di un'implementazione pubblica o di pseudo-codice formale nel lavoro originale, si è proceduto alla ricostruzione dell'architettura partendo dalle specifiche descrittive e dai vincoli teorici delineati dagli autori. Tale processo di *reverse engineering* è finalizzato a saggiare la robustezza e la riproducibilità dei risultati dichiarati. Successivamente, per testare la capacità di generalizzazione del modello, l'analisi è stata estesa a un portafoglio eterogeneo rispetto a quello originariamente esaminato. Infine, vengono discusse alcune soluzioni tecniche migliorative volte a raffinare la precisione della stima del VaR e a incrementare la fedeltà del modello alle dinamiche stocastiche dei mercati finanziari.

2 Analisi del Paper Originale

Il paper presenta il TempVAE per apprendere la distribuzione multivariata dei rendimenti degli asset. Si assume che la serie di rendimenti $R = R_{1:T}$ sia generata tramite variabili latenti $Z = Z_{1:T}$, dove $T \in \mathbb{N}$ e $T > 1$. Le distribuzioni delle variabili Z_t sono stimate tramite inferenza approssimata e ci aspettiamo che contengano informazioni come l'attuale contesto di mercato, la volatilità o le interazioni. Dati i due processi stocastici a tempo discreto R e Z , assumiamo una dipendenza temporale autoregressiva dalle informazioni latenti, data da:

$$Z_t | Z_{1:t-1} \sim \mathcal{N}(\mu^z(Z_{1:t-1}), \Sigma^z(Z_{1:t-1})) \quad (1)$$

$$R_t | Z_{1:t} \sim \mathcal{N}(\mu^r(Z_{1:t}), \Sigma^r(Z_{1:t})) \quad (2)$$

dove μ^z , μ^r , Σ^z e Σ^r sono funzioni che mappano rispettivamente $\mathbb{R}^\kappa$, $\mathbb{R}^d$, $\mathbb{R}^{\kappa \times \kappa}$ e $\mathbb{R}^{d \times d}$. L'equazione (1) descrive la dinamica temporale delle variabili nascoste. Ovvero lo stato latente al tempo t (Z_t) dipende dalla storia di tutti gli stati latenti precedenti ($Z_{1:t-1}$), dunque il modello assume che il "mood" del mercato di oggi non sia casuale, ma sia un'evoluzione di quello di ieri, un processo auto-regressivo. Matematicamente è una distribuzione Normale (Gaussiana) dove la media (μ_z) e la varianza (Σ_z) sono calcolate (tramite una rete neurale) basandosi sul passato. Mentre l'equazione (2) descrive l'emissione del dato reale, il rendimento dell'asset al tempo t (R_t) è generato in base alla sequenza delle variabili latenti fino a quel momento ($Z_{1:t}$). Data questa struttura di dipendenza, la distribuzione congiunta può essere fattorizzata come:

$$p_\theta(Z) = \prod_{t=1}^T p_\theta(Z_t | Z_{1:t-1}) \quad (3)$$

$$p_\theta(R|Z) = \prod_{t=1}^T p_\theta(R_t | Z_{1:t}) \quad (4)$$

Rispettivamente queste definizioni rappresentano la **Prior** e la **Likelihood** del modello. Il paper per tenere conto della dipendenza delle sequenze passate, fa utilizzo di strati RNN, più precisamente delle GRU (Gated Recurrent Unit). Inoltre utilizziamo perceptor multistrato (MLP), ovvero una rete Feed Forward fitta, per mappare l'output degli RNN allo spazio dei parametri delle distribuzioni. In totale, utilizziamo l'architettura del modello data da:

$$\{\mu_t^z, \Sigma_t^z\} = \text{MLP}_G^z(h_t^z) \quad (5)$$

$$h_t^z = \text{RNN}_G^z(h_{t-1}^z, Z_{t-1}) \quad (6)$$

$$Z_t \sim \mathcal{N}(\mu_t^z, \Sigma_t^z) \quad (7)$$

$$\{\mu_t^r, \Sigma_t^r\} = \text{MLP}_G^r(h_t^r) \quad (8)$$

$$h_t^r = \text{RNN}_G^r(h_{t-1}^r, Z_t) \quad (9)$$

$$R_t \sim \mathcal{N}(\mu_t^r, \Sigma_t^r) \quad (10)$$

Dopo aver definito dunque l'architettura, all'interno del testo si fa chiaro riferimento a una prior non addestrabile (vedere Appendice B.2 del paper). Le realizzazioni di Z influenzano le realizzazioni di R così come tutte le realizzazioni future di R e Z . Mentre le informazioni sui punti temporali precedenti vengono propagate attraverso gli stati nascosti RNN h^r e h^z . Modello di inferenza: si approssimiamo l'intrattabile $p_\theta(Z|R)$ tramite la distribuzione di inferenza $q_\phi(Z|R)$. Analogamente a Krishnan et al.

(2017), si può dimostrare che $Z_t \perp\!\!\!\perp R_{1:t-1} | Z_{1:t-1}$. Pertanto, si fattorizza la distribuzione in modo che la variabile Z_t dipenda dalle osservazioni (future) $R_{t:T}$:

$$p_\theta(Z|R) = \prod_{t=1}^T p_\theta(Z_t | Z_{1:t-1}, R_{t:T}) \quad (11)$$

Krishnan et al. (2017) sostengono che q dovrebbe essere fattorizzato allo stesso modo. Poiché q è solo un'approssimazione di p , si fornisce la sequenza completa che produce prestazioni migliori (vedi Appendice B.7). Pertanto si fattorizza q come:

$$q_\phi(Z|R) = \prod_{t=1}^T q_\phi(Z_t | Z_{1:t-1}, R_{1:T}) \quad (12)$$

e si assume $q_\phi(Z_t | Z_{1:t-1}, R_{1:T}) \sim \mathcal{N}(\hat{\mu}_z(Z_{1:t-1}, R_{1:T}), \hat{\Sigma}_z(Z_{1:t-1}, R_{1:T}))$. Qui $\hat{\mu}_z$ e $\hat{\Sigma}_z$ sono funzioni che mappano rispettivamente $\mathbb{R}^\kappa$ e $\mathbb{R}^{\kappa \times \kappa}$. Inoltre, la distribuzione dell'inferenza si implementa tramite un RNN bidirezionale come segue:

$$\{\hat{\mu}_t^z, \hat{\Sigma}_t^z\} = \text{MLP}_I^z(\hat{h}_t^z) \quad (13)$$

$$\hat{h}_t^z = \text{RNN}_I^z(\hat{h}_{t-1}^z, Z_{t-1}, [\hat{h}_t^{\rightarrow}, \hat{h}_t^{\leftarrow}]) \quad (14)$$

$$\hat{h}_t^{\rightarrow} = \text{RNN}_I^z(\hat{h}_{t-1}^{\rightarrow}, R_{t-1}) \quad (15)$$

$$\hat{h}_t^{\leftarrow} = \text{RNN}_I^z(\hat{h}_{t+1}^{\leftarrow}, R_{t+1}) \quad (16)$$

$$Z_t \sim \mathcal{N}(\hat{\mu}_t^z, \hat{\Sigma}_t^z) \quad (17)$$

Poiché si tratta di un'ipotesi di distribuzione con un $p(Z|X)$ intrattabile, il solito sospettato per l'addestramento del modello è il limite inferiore dell'evidenza (ELBO) dato come:

$$L(\theta, \phi) = \mathbb{E}_{q_\phi(Z|R)} [\log p_\theta(R | Z)] + \mathbb{E}_{q_\phi(Z|R)} \left[\log \left(\frac{p_\theta(Z)}{q_\phi(Z | R)} \right) \right] \quad (18)$$

$$= \mathbb{E}_{q_\phi(Z|R)} [\log p_\theta(R | Z)] - D_{\text{KL}}(q_\phi(Z | R) \| p_\theta(Z)) \quad (19)$$

Le componenti sono date dalle equazioni (3), (4) e (12). Poiché abbiamo $\log p_\theta(R) \geq \mathcal{L}(\theta, \phi)$, la massimizzazione dell'ELBO porta molto probabilmente a un aumento della verosimiglianza del nostro modello di distribuzione. In modo simile a Krishnan et al. (2017), possiamo riscrivere l'ELBO come:

$$\mathcal{L}(\theta, \phi) = \mathbb{E}_{q_\phi(Z|R)} [\log p_\theta(R|Z)] - \sum_{t=1}^T \mathbb{E}_{q_\phi(Z_{1:t-1}|R)} [D_{\text{KL}}(q_\phi(Z_t | Z_{1:t-1}, R) \| p_\theta(Z_t | Z_{1:t-1}))] \quad (20)$$

Poiché ipotizziamo una distribuzione gaussiana per q e p , possiamo calcolare analiticamente la divergenza di Kullback-Leibler (KL-Divergence) all'interno dell'aspettativa in (20). Approssimiamo numericamente le aspettative esterne tramite Monte Carlo. Implementiamo il modello in base alla Sezione 3. Per le RNN la scelta è ricaduta sulle 'Gated Recurrent Units' (come Cho et al (2014)) con dimensione 16 e per le MLP delle normali reti feedforward che hanno due strati nascosti con dimensione 16 e lo strato di output con dimensione di output variabile. Per le variabili latenti la dimensione sarà 10. Negli esperimenti eseguiti da Sicks et al., con questa dimensione della dimensione latente il modello è sovraspecificato e l'auto-pruning rimuove le dimensioni non necessarie. Tutti i pesi sono inizializzati da strati MLP con l'inizializzatore 'Variance Scaling' come proposto da He et al (2015). Come funzione di attivazione per gli strati nascosti utilizziamo la "Rectified Linear Unit" (ReLU). Per gli strati μ_z , $\hat{\mu}_z$ e μ_r non si utilizza alcuna attivazione. Una struttura di covarianza diagonale viene utilizzata per Σ_z e $\hat{\Sigma}_z$ e un output di perturbazione di rango 1 come indicato in

Rezende et al. (2014) per Σ_r . Per tutti gli strati di covarianza, si fa uso dell'attivazione esponenziale per gli elementi diagonali. Inoltre, presente la regolarizzazione L2 con parametro $\lambda = 0,01$ per i pesi degli strati nascosti dell'MLP, nonché un dropout con un tasso del 10% per gli strati RNN. Negli esperimenti, sia il dropout che la regolarizzazione L2 si sono rivelati cruciali per il corretto funzionamento dell'auto-pruning e per un'adeguata modellazione dei dati (vedere Appendice B.6).

3 Implementazione e Riproducibilità

Per replicare l'architettura presentata dal paper, la scelta è ricaduta su un'implementazione tramite Python, linguaggio più adatto allo sviluppo di tali reti complesse. Infatti questo linguaggio presenta diverse librerie adatte a tale scopo, tra cui PyTorch e TensorFlow, la prima di queste è quella utilizzata per sviluppare il codice relativo al TempVAE. Lo sviluppo avviene su un hardware compatibile con la libreria che dunque permette l'utilizzo della tecnologia Cuda, infatti quest'ultimo linguaggio gira su una GPU NVIDIA 4070 Super. Per replicare al meglio l'architettura sono state create 2 classi distinte: **TempVAE**, classe relativa all'intera architettura, e **MLP**, dove viene definita la parte di codice replicabile più volte.

3.1 MLP

L'architettura del TempVAE impiega una sottorete MLP (Multi-Layer Perceptron) come modulo funzionale ricorrente per la parametrizzazione delle distribuzioni latenti e condizionate. Dal punto di vista matematico, questa componente agisce come un approssimatore universale $f_\theta : \mathbb{R}^n \rightarrow \mathbb{R}^{2d}$ incaricato di mappare le rappresentazioni intermedie nei parametri di una densità Gaussiana multivariata a matrice di covarianza diagonale.

La rete è strutturata come una feed-forward profonda con due layer nascosti di dimensione costante ($h = 16$). Formalmente, dato un input x , la trasformazione segue la catena: **Hidden Layers**:

$$\begin{aligned} h_1 &= \text{ReLU}(W_1 x + b_1) \\ h_2 &= \text{ReLU}(W_2 h_1 + b_2) \end{aligned}$$

dove $W_i \in \mathbb{R}^{16 \times n_i}$ e l'attivazione $\text{ReLU}(z) = \max(0, z)$ introduce la non-linearità necessaria per catturare le dinamiche non gaussiane tipiche dei rendimenti finanziari.

Il modulo termina con due rami lineari paralleli che proiettano lo spazio latente h_2 nei parametri della distribuzione $q(z|x)$ o $p(x|z)$:

- **Vettore della Media** (μ^z): Rappresenta il parametro di locazione. Essendo lo spazio delle medie $\mathbb{R}^d$, non viene applicata alcuna funzione di attivazione, permettendo alla rete di traslare liberamente la distribuzione nel dominio reale.
- **Vettore di Covarianza** (Σ^z): In linea con l'ipotesi di indipendenza condizionale delle componenti latenti, la rete produce la diagonale della matrice di covarianza. Il valore restituito è in forma "grezza" (pre-activation): in fase di output verrà interpretato come $\log(\sigma^2)$ o passato attraverso una funzione softplus per garantirne la positività stretta.

Nella fase di inizializzazione, per mitigare il problema della scomparsa del gradiente e stabilizzare l'apprendimento nelle prime epoche, si adotta la He Initialization (o Variance Scaling, specificata nel paper). I pesi sono campionati da una distribuzione con varianza legata al numero di input unit (n_{in}):

$$\text{Var}(W) = \frac{2}{n_{in}}$$

```

class MLP(nn.Module):
    """
    Modulo MLP generico come descritto:
    - 2 hidden layers con dimensione 16
    - Attivazione ReLU
    - Inizializzazione 'Variance Scaling' (He Initialization)
    """
    def __init__(self, input_dim, latent_dim=10, hidden_dim=16):
        super(MLP, self).__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim),
            nn.ReLU(),
        )

        # 2. Media (nessuna attivazione, può essere negativa)
        self.fc_mu = nn.Linear(hidden_dim, latent_dim)

        # 3. Diagonale della Covarianza
        # output ancora "grezzo" (pre-attivazione)
        self.fc_cov_diag = nn.Linear(hidden_dim, latent_dim)

        # He Initialization (Variance Scaling)
        self.apply(self._init_weights)

```

Mentre la fase di forward del modulo MLP trasforma deterministicamente l'input x nei parametri di una densità di probabilità. Il processo si articola in due step:

1. **Feature Extraction:** L'input attraversa la cascata di trasformazioni affini e attivazioni non lineari ($h = \text{ReLU} \circ \text{Lin}_2 \circ \text{ReLU} \circ \text{Lin}_1(x)$). Questo proietta i dati in uno spazio a 16 dimensioni dove le relazioni tra le variabili vengono linearizzate.
2. **Parameter Mapping:** Il vettore latente h viene proiettato parallelamente su due teste lineari:
 - **Media μ :** stima il centro della massa di probabilità.
 - **Covarianza (Σ_{diag}):** Alla proiezione lineare ($W_\sigma h + b_\sigma$) viene applicata endogenamente una attivazione esponenziale ($\sigma = \exp(\text{raw_diag})$).

Dunque avremo un workflow di questa piccola rete che segue questa logica:

$$x \xrightarrow{\text{non-linear}} h_1 \xrightarrow{\text{non-linear}} h_2 \xrightarrow{\text{linear}} (\mu, \sigma)$$

```

def _init_weights(self):
    for m in self.net:
        if isinstance(m, nn.Linear):
            # He initialization (Variance Scaling)
            nn.init.kaiming_normal_(m.weight, mode='fan_in', nonlinearity='relu')
            nn.init.constant_(m.bias, 0)

def forward(self, x):
    h = self.net(x)

    # Calcolo della media
    mu = self.fc_mu(h)

    # Calcolo della diagonale grezza
    raw_diag = self.fc_cov_diag(h)

    # Exponential activation
    diag_entries = torch.exp(raw_diag)

    return mu, diag_entries

```

3.2 Encoder

In conformità con il paradigma del Variational Autoencoder, il modello di inferenza (o *Recognition Network*) ha il compito di approssimare la distribuzione a posteriori intrattabile $q_\phi(z_{1:T}|x_{1:T})$. A differenza delle architetture VAE statiche, la modellazione di serie storiche finanziarie richiede una gestione esplicita della dipendenza temporale; pertanto, l'architettura dell'encoder è progettata per implementare fedelmente le dinamiche descritte dalle Equazioni (13)-(16) del lavoro di riferimento. Il processo di inferenza si articola in tre fasi sequenziali:

1. **Estrazione del Contesto Bidirezionale (Eq. 15-16):** La prima componente è costituita da una Gated Recurrent Unit bidirezionale (BiGRU). Questa rete processa l'intera sequenza dei rendimenti osservati R , generando per ogni istante t una rappresentazione contestuale data dalla concatenazione degli stati nascosti forward e backward: $[\vec{h}_t, \overleftarrow{h}_t]$. Matematicamente, questa operazione trasforma i dati grezzi in un embedding che incorpora informazioni sia passate che future (*smoothing*), essenziali per catturare trend globali e reagire a shock strutturali improvvisi nel mercato.
2. **Inferenza Autoregressiva (Eq. 14):** Il cuore del meccanismo di inferenza è una cella ricorrente (GRUCell) che fonde il contesto osservato con la memoria dello stato latente. Come specificato nel paper, l'input a questo blocco è un vettore composito definito da:

$$\text{Input}_t = [\hat{h}_{t-1}^z, Z_{t-1}, \text{Context}_t]$$

dove Context_t è l'output della BiGRU. Questa struttura autoregressiva assicura che la stima della distribuzione corrente sia condizionata coerentemente dall'evoluzione storica del processo latente.

3. **Mappatura dei Parametri e Regularizzazione (Eq. 13):** Infine, lo stato nascosto risultante viene proiettato, mediante un modulo MLP, nello spazio dei parametri μ_q e σ_q che definiscono la densità variazionale gaussiana. Per garantire la robustezza del modello e mitigare il rischio di overfitting, viene applicato uno schema di **dropout** all'interno dei layer ricorrenti; questa tecnica di regularizzazione forza la rete a sviluppare rappresentazioni ridondanti e resilienti, migliorando la capacità di generalizzazione sui dati fuori campione.

```
class TempVAE(nn.Module):
    def __init__(self, input_dim, latent_dim=10, hidden_dim=16, dropout=0.1):
        """
        Args:
            input_dim (int): Dimensione dei rendimenti degli asset (d)
            latent_dim (int): Dimensione dello spazio latente (kappa)
            hidden_dim (int): Dimensione hidden per RNN e MLP (default 16)
            dropout (float): Rate di dropout per le RNN (default 10%)
        """
        super(TempVAE, self).__init__()

        self.input_dim = input_dim
        self.latent_dim = latent_dim
        self.hidden_dim = hidden_dim

        # --- INFERENCE MODEL (Encoder) ---
        # Eq (15) & (16): Bidirectional RNN per estrarre il contesto da R
        # Input: R_t, Output: [h_forward, h_backward] (dimensione 2 * hidden_dim)
        self.lnf_bi_rnn = nn.GRU(input_dim, hidden_dim, bidirectional=True, batch_first=True)
        self.lnf_dropout = nn.Dropout(dropout)

        # Eq (14): RNN inferenza autoregressiva
        # Input: concat(h^z_{t-1}, Z_{t-1}, Context_t)
        # Context_t è l'output della BiRNN (2*hidden)
        self.lnf_rnn_z = nn.GRUCell(latent_dim + 2 * hidden_dim, hidden_dim)

        # Eq (13): MLP per parametri q(Z)
        # Output: mu (kappa) + sigma_diag (kappa)
        self.lnf_mlp = MLP(hidden_dim, latent_dim, hidden_dim)
```


3.3 Decoder

Il blocco generativo, speculare all'encoder, è preposto alla ricostruzione della distribuzione congiunta $p_\theta(R_{1:T}, Z_{1:T})$. Architetturealmente, questo modulo si biforca in due sottoprocessi distinti che operano in tandem: la modellazione della dinamica latente (Prior) e la generazione dei dati osservabili (Modello di Emissione).

1. **Dinamica Latente e Prior "Congelata" (Eq. 5-6):** La legge di evoluzione dello stato latente $p_\theta(Z_t|Z_{t-1})$ è governata da una cella ricorrente (`gen_rnn_z`) seguita da un MLP (`gen_mlp_z`). Una peculiarità critica dell'implementazione, derivata dall'Appendice B.2 del paper originale, è la scelta di utilizzare una **Prior non addestrabile**. Nel codice, questo si traduce nell'inizializzazione dei pesi tramite *He Initialization* e nel successivo congelamento dei gradienti (`requires_grad = False`). Questa scelta architettureale impone che lo spazio latente si conformi a una dinamica stocastica prefissata e casuale. Tale vincolo agisce come un potente regolarizzatore: previene il fenomeno del *Posterior Collapse* (dove il decoder ignora la variabile latente) e forza l'encoder a mappare le osservazioni in uno spazio con traiettorie dinamiche ben definite, migliorando la separabilità temporale delle feature.
2. **Modello di Emissione (Eq. 9):** La probabilità condizionata dei dati osservati $p_\theta(R_t|Z_{1:t})$ è modellata da una seconda catena ricorrente (`gen_rnn_r`). Lo stato nascosto h_t^r funge da memoria del processo generativo, aggiornandosi passo dopo passo in base al nuovo stato latente campionato Z_t . Questo assicura che la generazione del rendimento attuale sia coerente con la storia evolutiva del sistema.
3. **Parametrizzazione Low-Rank della Covarianza (Eq. 8):** L'aspetto più sofisticato del decoder risiede nella proiezione finale (`gen_mlp_r`). Per stimare la matrice di covarianza $\Sigma_t^r \in \mathbb{R}^{d \times d}$ in uno spazio ad alta dimensionalità (d), il modello evita la computazione quadratica $O(d^2)$ di una matrice piena. Si adotta invece una perturbazione di **Rango-1**, dove l'output della rete è un vettore concatenato di dimensione $3d$, contenente:

- Il vettore delle medie $\mu_r \in \mathbb{R}^d$.
- Il vettore della diagonale $D \in \mathbb{R}_+^d$.
- Il vettore fattore $u \in \mathbb{R}^d$.

La matrice di covarianza risultante è calcolata algebricamente come:

$$\Sigma_t^r = \text{diag}(D_t) + u_t u_t^T$$

```

# --- GENERATIVE MODEL (Decoder) ---
# Eq (6): RNN Prior
# Input: Z_{t-1}, Hidden: h^z_{t-1}
self.gen_rnn_z = nn.GRUCell(latent_dim, hidden_dim)

# Eq (5): MLP Prior
# Output: mu (kappa) + sigma_diag (kappa)
self.gen_mlp_z = MLP(hidden_dim, latent_dim, hidden_dim)

# Inizializzazione e Congelamento (Freeze) per Prior Fissa
for m in [self.gen_rnn_z, self.gen_mlp_z]:
    # Inizializzazione He
    for name, param in m.named_parameters():
        if 'weight' in name:
            nn.init.kaiming_normal_(param.data)
        elif 'bias' in name:
            nn.init.zeros_(param.data)

    # Congeliamo i pesi
    for param in m.parameters():
        param.requires_grad = False

# Eq (9): RNN Emissione
# Input: Z_t, Hidden: h^r_{t-1}
self.gen_rnn_r = nn.GRUCell(latent_dim, hidden_dim)

# Eq (8): MLP Emissione
# Output per Rank-1 Perturbation:
# Diagonale D (input_dim)
# Vettore fattore u (input_dim) -> per uu^Trasposta
# Media mu (input_dim)
self.gen_mlp_r = nn.Sequential(
    nn.Linear(hidden_dim, hidden_dim),
    nn.ReLU(),
    nn.Linear(hidden_dim, input_dim * 3)
)
self.gen_dropout = nn.Dropout(dropout)

```

3.4 Forward Pass e Calcolo della Loss

La funzione `forward` costituisce il nucleo operativo dell'architettura durante la fase di addestramento. Essa orchestra l'intero ciclo computazionale, traducendo le relazioni di ricorrenza teoriche in operazioni vettorializzate efficienti per il calcolo simultaneo della Loss di Ricostruzione e della Divergenza KL. Il flusso di esecuzione si articola in tre stadi logici distinti:

1. **Pre-elaborazione del Contesto (Context Shifting):** Prima di avviare il ciclo ricorrente, il modello elabora l'intera sequenza temporale $R_{1:T}$ attraverso la BiGRU. Un passaggio critico, spesso trascurato nelle implementazioni standard ma essenziale per rispettare la causalità temporale definita dalle Eq. (15) e (16), è lo *shifting* degli stati nascosti.

- Lo stato *forward* $\hat{h}_t^{\rightarrow}$ deve incapsulare le informazioni fino a $t - 1$.

- Lo stato *backward* $\hat{h}_t^{\leftarrow}$ deve aggregare le informazioni future da $t + 1$.

Nel codice, questo viene realizzato mediante operazioni di *padding* e *slicing* sui tensori:

$$\text{Context}_t = [\hat{h}_{t-1}^{\rightarrow}, \hat{h}_{t+1}^{\leftarrow}]$$

Questa manipolazione assicura che, al tempo t , la rete di inferenza non abbia accesso "improprio" all'osservazione corrente R_t attraverso il contesto, preservando la struttura autoregressiva del modello generativo.

2. **Ciclo Ricorrente Unificato (Training Loop):** Il modello itera sequenzialmente su ogni passo temporale t . All'interno di ogni iterazione, vengono eseguiti congiuntamente i passi di inferenza e generazione:

- **Inferenza Variazionale $q(Z_t|\cdot)$:** La GRU di inferenza combina il contesto "shiftato" pre-calcolato con lo stato latente precedente Z_{t-1} , producendo i parametri μ_q, Σ_q . Da questa distribuzione viene campionato Z_t tramite il *reparameterization trick*.
- **Aggiornamento della Prior $p(Z_t|\cdot)$:** Parallelamente, la rete generativa della prior (congelata) aggiorna il proprio stato interno basandosi su Z_{t-1} , fornendo i parametri di riferimento μ_p, Σ_p per il calcolo della KL.
- **Generazione dell'Output $p(R_t|\cdot)$:** Infine, lo stato latente attuale Z_t alimenta la GRU di emissione, che stima la densità di probabilità del rendimento corrente R_t .

3. **Funzione Obiettivo (ELBO):** La loss function finale è composta dalla somma di due termini, accumulati lungo l'asse temporale:

- **Negative Log-Likelihood (NLL):** Misura l'errore di ricostruzione. Sfruttando la classe `LowRankMultivariateNormal` di PyTorch, calcoliamo efficientemente $\log p(R_t|\mu_r, D, u)$ senza dover invertire esplicitamente matrici dense, massimizzando la probabilità che i dati osservati siano stati generati dal modello.
- **KL Divergence Analitica:** Calcoliamo la divergenza $D_{KL}(q(Z_t)||p(Z_t))$ in forma chiusa per due multivariate gaussiane diagonali. Questo termine penalizza la distribuzione a posteriori se devia troppo dalla prior dinamica, agendo come regolarizzatore bayesiano.

$$\mathcal{L} = \sum_{t=1}^T (-\log p_{\theta}(R_t|Z_{1:t}) + D_{KL}(q_{\phi}(Z_t|\cdot)||p_{\theta}(Z_t|\cdot)))$$

```

def forward(self, x):
    """
    Args:
        x: Tensore [Batch, Seq_Len, Input_Dim] (R_{1:T})
    Returns:
        recon_loss: Negativa Log-Likelihood
        kl_loss: KL Divergence totale
    """
    batch_size, seq_len, _ = x.size()
    device = x.device

    # --- 1. Bi-RNN Standard su R ---
    # Output shape: [Batch, Seq, 2*Hidden]
    # Contiene [h_fwd_t, h_bwd_t] allineati con R_t
    bi_rnn_out_raw, _ = self.inf_bi_rnn(x)
    bi_rnn_out_raw = self.inf_dropout(bi_rnn_out_raw)

    # Separazione h forward e backward
    # fwd_out[t] contiene info su R_0...R_t
    # bwd_out[t] contiene info su R_t...R_T
    fwd_out = bi_rnn_out_raw[:, :, :self.hidden_dim]
    bwd_out = bi_rnn_out_raw[:, :, self.hidden_dim:]

    # --- 2. Implementazione Shifting (Eq 15 e 16) ---

    # Eq (15): h_forward_t deve dipendere da R_{t-1}.
    # Al tempo t=0, non c'è R_{-1}, quindi usiamo uno stato zero (padding).
    # Al tempo t=k, usiamo fwd_out[k-1].
    zeros_fwd = torch.zeros(batch_size, 1, self.hidden_dim).to(device)
    h_arrow_right = torch.cat([zeros_fwd, fwd_out[:, :-1, :]], dim=1)

    # Eq (16): h_backward_t deve dipendere da R_{t+1}.
    # Al tempo t=T-1 (ultimo), non c'è R_T, usiamo zero.
    # Al tempo t=k, usiamo bwd_out[k+1].
    zeros_bwd = torch.zeros(batch_size, 1, self.hidden_dim).to(device)
    h_arrow_left = torch.cat([bwd_out[:, 1:, :], zeros_bwd], dim=1)

    # Concateniamo i contesti corretti per l'equazione (14)
    # context_shifted contiene esattamente [h->_{t}, h<-_{t}] da Eq 15-16
    context_shifted = torch.cat([h_arrow_right, h_arrow_left], dim=2)

    # ... Inizializzazione stati ...
    h_z = torch.zeros(batch_size, self.hidden_dim).to(device)
    # h_gen_z per la Prior (inizializzato a zero)
    h_gen_z = torch.zeros(batch_size, self.hidden_dim).to(device)
    h_gen_r = torch.zeros(batch_size, self.hidden_dim).to(device)
    z_prev = torch.zeros(batch_size, self.latent_dim).to(device)

    kl_loss = 0
    nll_loss = 0

    for t in range(seq_len):

        # Eq (14): RNN_I^*z prende (h_{t-1}, Z_{t-1}), [h->_t, h<-_t])
        # Usiamo il contesto shifted calcolato sopra
        current_context = context_shifted[:, t, :] # [Batch, 2*Hidden]

        rnn_input_inf = torch.cat([z_prev, current_context], dim=1)
        h_z = self.inf_rnn_z(rnn_input_inf, h_z)
        h_z = self.inf_dropout(h_z)

        # Eq (13): Ottieni parametri q(Z_t | ...)
        q_mu, q_sigma = self.inf_mlp(h_z)

        # --- GENERATIVE PRIOR (p(Z)) ---
        # Prior: p(Z_t | Z_{1:t-1})
        # Aggiornamento hidden state usando Z_{t-1} (z_prev)
        h_gen_z = self.gen_rnn_z(z_prev, h_gen_z)

        # Calcolo parametri prior per lo step t
        p_mu, p_sigma = self.gen_mlp_z(h_gen_z)

        # Calcolo KL Divergence analitica per step t (Eq 20 interna)
        # D_KL(N(q_mu, q_var) || N(p_mu, p_var))
        q_dist = dist.Normal(q_mu, q_sigma)
        p_dist = dist.Normal(p_mu, p_sigma)
        kl_loss += dist.kl_divergence(q_dist, p_dist).sum(dim=1).mean()

        # Sample Z_t (Eq 17)
        z_t = q_dist.rsample()

        # --- GENERATIVE EMISSION (p(R|Z)) ---
        # Eq (9): Aggiorno stato nascosto generazione usando Z_t ATTUALE
        h_gen_r = self.gen_rnn_r(z_t, h_gen_r)

        h_gen_r = self.gen_dropout(h_gen_r)

        # Eq (8): Parametri generazione
        r_params = self.gen_mlp_r(h_gen_r)

        # Decomposizione parametri per Rank-1 Covariance (LowRankMultivariateNormal)
        # mu: [Batch, d]
        # diag_log: [Batch, d] (log della diagonale)
        # factor: [Batch, d] (vettore u per uu^T)
        r_mu = r_params[:, :self.input_dim]
        r_diag_log = r_params[:, self.input_dim : 2*self.input_dim]
        r_factor = r_params[:, 2*self.input_dim :].unsqueeze(-1) # [Batch, d, 1]

        # Attivazione esponenziale per la diagonale
        r_diag = torch.exp(r_diag_log) + 1e-4

        # Distribuzione Multivariate Normal con struttura Low Rank
        r_dist = dist.LowRankMultivariateNormal(
            loc=r_mu,
            cov_factor=r_factor,
            cov_diag=r_diag
        )

        # Calcolo Log Likelihood (Eq 18 -> Eq 19)
        # MASSIMIZZARE la log prob. quindi per la loss prendiamo il negativo
        nll_loss -= r_dist.log_prob(x[:, t, :]).mean()

        # Preparazione per il prossimo step
        z_prev = z_t

    return nll_loss, kl_loss

```

4 Risultati Sperimentali

In questa sezione vengono presentati i risultati dell’analisi empirica condotta per validare l’architettura Temporal VAE. L’obiettivo è valutare la capacità del modello di stimare accuratamente il Value-at-Risk (VaR) su dati finanziari reali, verificandone la robustezza attraverso test statistici di backtesting standard.

4.1 Setup Sperimentale

L’analisi empirica è stata articolata in due fasi complementari. In una prima fase, è stata condotta una replica quanto più fedele possibile del setting proposto nel paper di riferimento, utilizzando dati comparabili e lo stesso intervallo temporale, al fine di ottenere un confronto diretto e verificare la coerenza dei risultati.

In una seconda fase, l’attenzione si è spostata su una validazione più stringente: il modello è stato testato su un universo di asset più ampio e rappresentativo e su un orizzonte temporale significativamente più esteso, con l’obiettivo di valutarne la robustezza e la capacità di generalizzare a regimi di mercato differenti.

Per garantire la comparabilità con la letteratura di riferimento, il dataset, le procedure di addestramento e i parametri sono stati selezionati per rispecchiare fedelmente le dinamiche di mercato, mantenendo al contempo un costo computazionale sostenibile. Per quanto riguarda lo **Split Temporale**, il dataset è stato suddiviso cronologicamente in: 66% dei dati costituiscono il *Training Set*, mentre il restante 34% è utilizzato come *Test Set* per la valutazione fuori campione. I prezzi di chiusura aggiustati (P_t) sono stati trasformati in log-rendimenti (r_t) per garantire la stazionarietà della serie:

$$r_t = \ln(P_t) - \ln(P_{t-1})$$

Successivamente, i dati sono stati organizzati in finestre scorrevoli (*sliding windows*) di lunghezza M giorni, definita da un iperparametro apposito. Ogni input al modello è dunque un tensore di dimensione (B, M, D) , dove B è la dimensione del batch e D il numero di asset.

4.1.1 Dataset e Preprocessing

La prima analisi è stata condotta su un paniere di asset differente rispetto a quello considerato nel lavoro originale. Tale scelta è motivata dal fatto che il portafoglio utilizzato dagli autori è specifico e la selezione dei titoli del DAX non è riportata in modo univoco. Di conseguenza, l’esperimento è stato replicato su un portafoglio equiponderato e più generalizzabile, composto da un benchmark azionario globale (SPY), dall’oro (GLD) e dall’indice di volatilità implicita $\hat{VIX}$, così da aumentare l’eterogeneità del campione e catturare in maniera più informativa le dinamiche di dipendenza, soprattutto nei regimi di stress. È stato inoltre incluso un asset ad elevata volatilità (BTC) al fine di stressare ulteriormente la capacità del modello di adattarsi a distribuzioni con code pronunciate. Il periodo di osservazione si estende dal 2 ottobre 2018 al 30 giugno 2021, per rimarcare, almeno su questo aspetto, i dati dell’analisi di Sicks et al.

Nella seconda fase, il modello è stato testato su un universo multi-asset più ampio e rappresentativo e su un orizzonte temporale significativamente più esteso (2004–2025), con l’obiettivo di valutarne la robustezza e la capacità di generalizzare a regimi di mercato differenti. In particolare, la seconda analisi considera un paniere composto da: SPY (S&P 500), QQQ (Nasdaq 100), IWM (Russell 2000), EEM (Emerging Markets), XLF (Financials), XLE (Energy), TLT (Treasury 20+ Year), GLD (Gold) e $\hat{VIX}$ (volatilità implicita).

4.1.2 Protocollo di Addestramento

L’addestramento è avvenuto minimizzando la funzione di costo ELBO. Un aspetto critico per la convergenza del VAE è il bilanciamento tra il termine di ricostruzione (NLL) e la divergenza KL.

A tale scopo, è stata adottata la strategia di β -Annealing: il peso β della componente KL viene incrementato gradualmente da 0 a 1 durante le epoche, secondo la legge:

$$\beta_{epoch} = \max\left(0, 1 - \gamma^{\frac{epoch}{K}}\right)$$

dove $\gamma = 0.96$ è il tasso di decadimento e $K = 20$ il numero di step di annealing. Questo permette all'encoder di apprendere rappresentazioni significative prima che la regolarizzazione della prior diventi troppo stringente. L'ottimizzatore utilizzato è Adam con un learning rate iniziale di 10^{-3} , soggetto a decadimento esponenziale, e una regolarizzazione L2 ($\lambda = 0.01$) sui pesi della rete.

4.2 Metodologia di Stima e Backtesting

Una volta addestrato il modello, la stima del VaR giornaliero avviene attraverso una procedura di simulazione Monte Carlo che sfrutta la natura generativa del TempVAE.

4.2.1 Stima del VaR tramite Simulazione Monte Carlo

Dato un input storico $x_{t-M+1:t}$, il processo di stima per il giorno $t + 1$ si articola nei seguenti passi:

1. **Inferenza dello Stato Latente:** Il modello di inferenza (Encoder) processa la finestra temporale di input e produce la distribuzione a posteriori dell'ultimo stato latente $Z_t \sim q_\phi(Z_t|x)$. Da questa distribuzione viene estratto il valore atteso (o un campione rappresentativo) $\hat{z}_t$.
2. **Generazione degli Scenari:** Lo stato latente $\hat{z}_t$ viene passato al Decoder (Modello di Emissione), che restituisce i parametri della distribuzione predittiva dei rendimenti $p_\theta(R_{t+1}|\hat{z}_t) = \mathcal{N}(\mu_r, \Sigma_r)$.
3. **Campionamento:** Vengono generati $N = 1000$ scenari sintetici campionando dalla distribuzione normale multivariata parametrizzata da μ_r e Σ_r .
4. **Calcolo del VaR:** Per ogni scenario simulato, si calcola il rendimento del portafoglio (assumendo pesi equiprobabili $1/D$). Il VaR al livello di confidenza α (es. 5%) è calcolato come il percentile α -esimo della distribuzione empirica dei rendimenti simulati:

$$\text{VaR}_{\alpha,t+1} = \text{percentile}(\{r_{sim}^{(i)}\}_{i=1}^N, \alpha)$$

4.2.2 Test Statistici di Validazione

La validità delle stime del VaR viene verificata mediante due test statistici fondamentali:

- **Kupiec POF Test (Proportion of Failures):** Verifica l'*accuratezza incondizionata* del modello. L'ipotesi nulla H_0 afferma che la frequenza osservata delle violazioni (giorni in cui la perdita reale supera il VaR) è statisticamente uguale al livello di confidenza teorico α . La statistica di test è un rapporto di verosimiglianza (Likelihood Ratio, LR) distribuito come un $\chi^2(1)$.
- **Christoffersen Conditional Coverage Test:** Estende il test di Kupiec verificando l'*indipendenza* delle violazioni. Un buon modello di rischio non dovrebbe presentare violazioni raggruppate nel tempo (clustering della volatilità). Il test combina la verifica della copertura incondizionata e quella dell'indipendenza seriale.

4.3 Analisi dei Risultati

L'analisi inizia adottando il modello, e gli iperparametri, così come definito nel paper di riferimento e seguendo la prima fase di costruzione del dataset, in modo da replicare il setting sperimentale originale e disporre di un benchmark coerente.

I risultati dei test di backtesting sul test set sono riassunti di seguito. Il livello di confidenza fissato per il VaR è il 95% ($\alpha = 0.05$).

Metrica	Valore Osservato	Target
Giorni Totali	322	322
Violazioni Osservate	17	16
Frequenza di Violazione	5.28%	5.00%

Tabella 1: Riepilogo delle Violazioni del VaR (95%)

Dalla Tabella 1 si nota che il numero di violazioni osservate (17) è estremamente vicino al valore atteso teorico (16.1), indicando una calibrazione eccellente del modello. Questa osservazione è formalmente confermata dai test statistici:

- **Kupiec POF Test:**

- LR Statistic: 0.0520
- P-Value: **0.8195**

Poiché il P-Value è ampiamente superiore alla soglia di significatività (0.05), **non rifiutiamo l'ipotesi nulla**. Ciò implica che il modello stima correttamente la frequenza delle perdite estreme.

- **Christoffersen Test (Copertura Condizionale):**

- LR Conditional Coverage: 0.0640
- P-Value: **0.9685**

Anche in questo caso, l'alto P-Value porta al **non rifiuto dell'ipotesi nulla**. Il test conferma non solo che il numero di violazioni è corretto, ma anche che esse non sono correlate serialmente (assenza di clustering). Il modello riesce dunque ad adattarsi rapidamente ai cambiamenti di volatilità del mercato, evitando periodi di sottostima sistematica del rischio.

In conclusione, l'analisi sperimentale conferma che l'architettura Temporal VAE implementata è statisticamente valida per la stima del rischio di mercato al 95% di confidenza, dimostrando capacità di generalizzazione su dati non visti durante l'addestramento.

4.4 Analisi plot

Analizzando i plot emergono indicazioni rilevanti circa la struttura dello spazio latente appresa dal modello. In particolare, il TempVAE tende a utilizzare un numero limitato di dimensioni latenti per rappresentare l'informazione contenuta nella serie temporale. Tale comportamento potrebbe ricordare un *posterior collapse*; tuttavia, l'evidenza empirica suggerisce che alcune unità rimangano effettivamente attive e interpretabili.

In un contesto finanziario, ci si attenderebbe tipicamente che 3–4 dimensioni latenti catturino fattori economici distinti (ad esempio volatilità, liquidità e condizioni macro). I risultati osservati indicano invece una rappresentazione più concentrata, con poche componenti dominanti (come da Figura 1) e altre a contributo marginale, come discusso nel seguito.

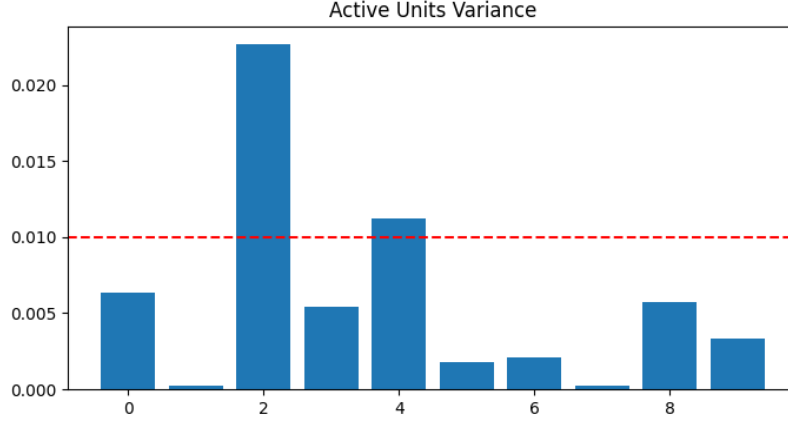


Figura 1: Unità latenti attive del modello.

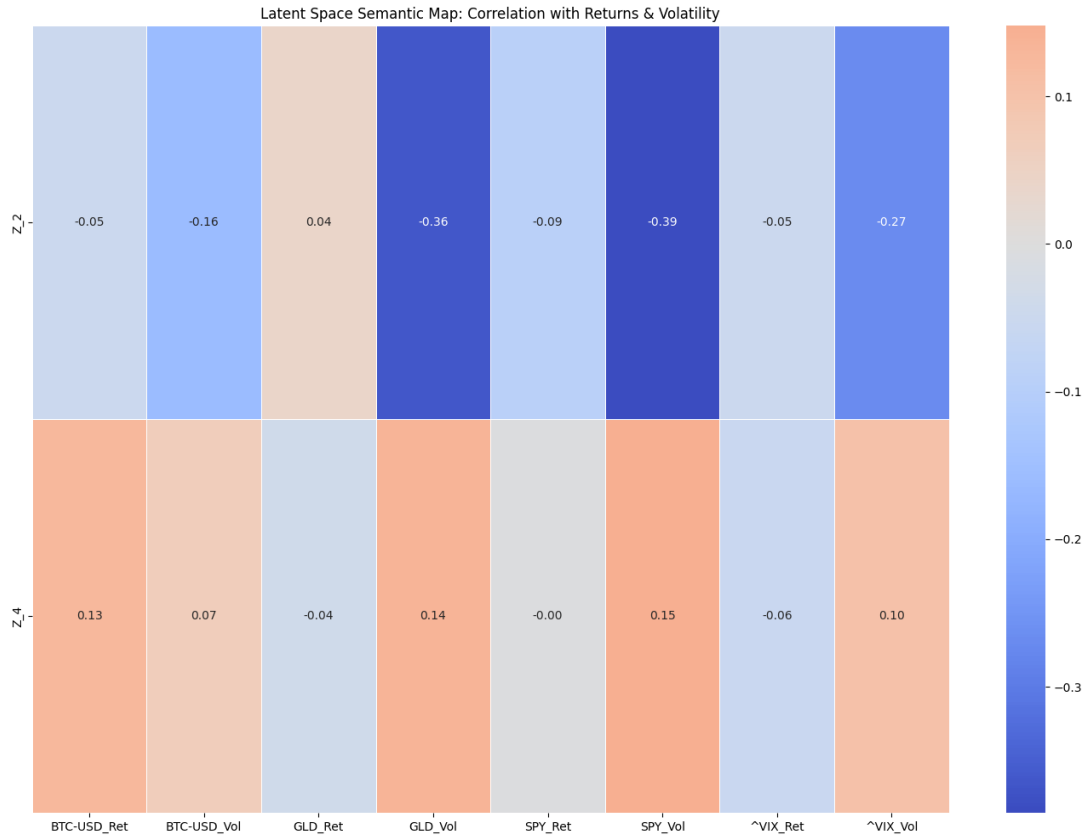
4.4.1 Latent Space Semantic Map e interpretazione delle componenti latenti

In questa sezione si analizza la *Latent Space Semantic Map*, una rappresentazione che quantifica la relazione tra le feature osservate X_i e le componenti latenti Z_j . In particolare, la mappa riporta la correlazione di Pearson (o, in termini più generali, una misura di sensibilità locale) tra ciascuna variabile osservata e la rappresentazione latente appresa dal modello.

Formalmente, consideriamo una matrice $\mathbf{H} \in \mathbb{R}^{D \times K}$, dove D è il numero di feature (nel nostro caso $D = 8$) e K la dimensionalità dello spazio latente. L'elemento (i, j) è definito come

$$\rho_{i,j} = \frac{\text{Cov}(X_i, \mu_{Z_j}(X))}{\sigma_{X_i} \sigma_{\mu_{Z_j}}},$$

ossia la correlazione tra la feature X_i e la media a posteriori della variabile latente Z_j . Valori $|\rho_{i,j}|$ elevati indicano che Z_j è sensibile alla variazione di X_i e quindi tende a codificare informazione associata a tale feature; valori prossimi a zero suggeriscono invece invarianza e, di conseguenza, un maggiore grado di *disentanglement*. Il segno di $\rho_{i,j}$ specifica la direzione della relazione.



Nel seguito si considerano, a titolo esemplificativo, i vettori Z_2 e Z_4 rispetto al seguente ordinamento delle feature: [BTC_Ret, BTC_Vol, GLD_Ret, GLD_Vol, SPY_Ret, SPY_Vol, VIX_Ret, VIX_Vol].

Analisi di Z_2 (fattore di volatilità sistemica). Per Z_2 si osserva il vettore di associazioni [-0.05, -0.16, -0.04, -0.36, -0.09, -0.39, -0.05, -0.27]. Le componenti con magnitudo maggiore sono concentrate sulle variabili di volatilità, in particolare **SPY_Vol** (-0.39), **GLD_Vol** (-0.36) e **VIX_Vol** (-0.27), mentre le correlazioni con le variabili di rendimento risultano prossime allo zero. Questo pattern suggerisce che Z_2 rappresenti un fattore latente interpretabile come regime di volatilità di mercato: esso è sostanzialmente disaccoppiato dalla componente direzionale dei rendimenti ed è invece guidato prevalentemente dalla dinamica della volatilità cross-asset.

Il segno negativo comune alle principali componenti indica una relazione inversa tra Z_2 e il livello di volatilità: valori più elevati di Z_2 sono associati a condizioni di maggiore stabilità (volatilità più bassa), mentre valori più negativi corrispondono a fasi di turbolenza sistemica in cui le volatilità aumentano simultaneamente. È inoltre rilevante che SPY_Vol e GLD_Vol presentino intensità comparabili, evidenziando la presenza di shock di volatilità trasversali che impattano sia l'equity sia l'asset considerato *safe haven*.

Analisi di Z_4 (fattore residuo a bassa intensità). Per Z_4 si ottiene il vettore [0.13, 0.07, -0.04, 0.14, -0.00, 0.15, -0.06, 0.10]. In questo caso le magnitudini sono significativamente inferiori rispetto a Z_2 (massimo pari a 0.15), suggerendo che Z_4 catturi una quota più limitata di varianza e possa configurarsi come unità latente secondaria. Il pattern risulta inoltre meno “pulito”: oltre alle associazioni positive con misure di volatilità (SPY_Vol, GLD_Vol, VIX_Vol), emerge una correlazione positiva con **BTC_Ret** (0.13), a fronte di una sostanziale indipendenza da SPY_Ret.

Complessivamente, Z_4 può essere interpretato come un fattore residuo compatibile con uno scenario di *risk-on* speculativo in condizioni di volatilità moderatamente crescente, in cui Bitcoin mostra performance positiva. Tuttavia, data la bassa ampiezza delle correlazioni (< 0.20), tale

interpretazione va considerata con cautela, poiché il segnale potrebbe riflettere componenti di non-linearità difficilmente catturabili da una metrica lineare come la correlazione.

4.4.2 Analisi temporale del modello

L'ultima visualizzazione consente un backtesting visivo delle stime di VaR. In un contesto di gestione del rischio, l'obiettivo non è riprodurre il valore medio dei rendimenti, bensì caratterizzare correttamente la coda sinistra della distribuzione, ovvero la regione associata alle perdite estreme. Il grafico sovrappone due serie temporali: i rendimenti realizzati R_t e la frontiera dinamica del Value-at-Risk, VaR_t , interpretata come soglia di perdita al livello di confidenza prefissato.

Un modello correttamente calibrato produce violazioni, definite come eventi $R_t < VaR_t$, con frequenza empirica prossima al livello α definito. A fini diagnostici, l'analisi del grafico può essere articolata lungo quattro dimensioni principali.

(A) Tasso di violazione (hit ratio). Si valuta la proporzione di osservazioni per cui $R_t < VaR_t$. Un numero di violazioni superiore a α segnala sottostima del rischio (code più pesanti rispetto a quanto modellato), mentre un numero inferiore indica un modello eccessivamente conservativo.

(B) Reattività ai cambi di volatilità. Si osserva la capacità della soglia VaR_t di adattarsi rapidamente dopo shock negativi. In presenza di un cambio di regime verso alta volatilità, un modello adeguato dovrebbe ampliare prontamente la banda di rischio (valori di VaR più negativi), minimizzando il ritardo di adattamento.

(C) Indipendenza delle violazioni (clustering). Oltre al conteggio complessivo, è rilevante la distribuzione temporale delle violazioni. La presenza di clustering (violazioni concentrate in brevi intervalli) suggerisce una modellazione incompleta della dinamica eteroschedastica, anche quando la frequenza totale risulta corretta.

(D) Severità delle violazioni. Infine, si considera la profondità delle violazioni, ossia l'entità di $R_t - VaR_t$ nei giorni in cui la soglia viene superata. Violazioni molto profonde e rare possono indicare che la distribuzione predittiva abbia code troppo leggere, evidenziando potenziali benefici nell'adozione di modelli con maggiore flessibilità sulle code (ad esempio distribuzioni heavy-tailed) e nel confronto con metriche complementari quali l'Expected Shortfall.

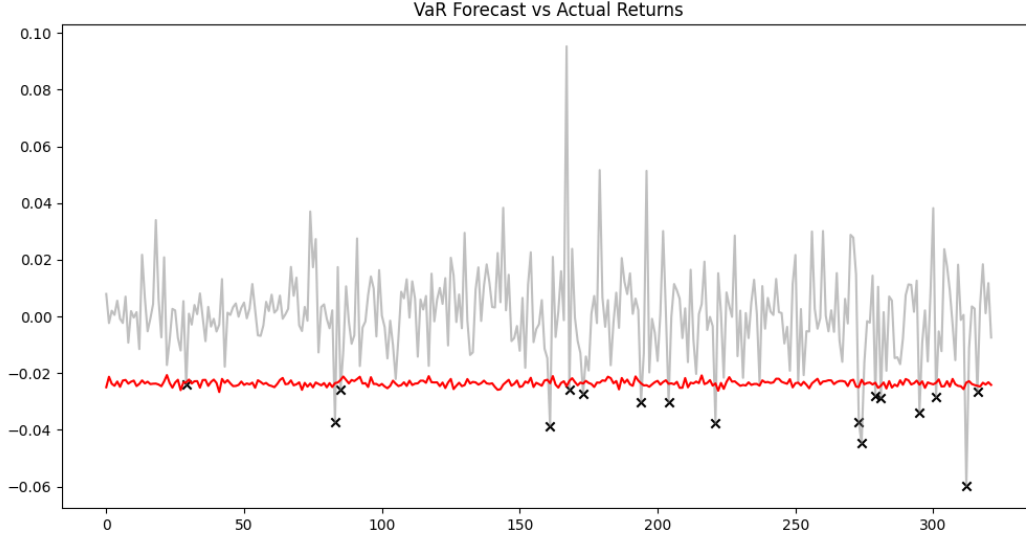


Figura 2: Backtesting visivo del Value-at-Risk: rendimenti realizzati R_t e soglia dinamica VaR_t (periodo 2018-2021, come da paper).

Nonostante il modello superi i test quantitativi riportati in precedenza e mostri una struttura latente complessivamente interpretabile, l'ispezione del backtesting visivo (Figura 2) evidenzia alcune criticità nella dinamica della soglia VaR_t . In particolare, in corrispondenza di shock negativi repentini il modello tende a reagire in modo non sempre coerente: in prossimità della prima violazione (indicata con X) la soglia aumenta invece di diventare più conservativa, e un comportamento analogo si osserva in presenza delle successive spike.

Al contrario, in corrispondenza di movimenti più moderati (ad esempio attorno al giorno 170) si osserva un irrigidimento della soglia, suggerendo una possibile asimmetria nella capacità di adattamento tra eventi estremi e variazioni meno pronunciate. Questo pattern è compatibile con una risposta eccessivamente “reattiva” a fluttuazioni moderate e, al contempo, insufficiente a catturare immediatamente cambi di regime caratterizzati da drawdown improvvisi.

La problematica più grossa però nasce nell'analisi di tale modello, con gli iperparametri definiti da Sicks et al., nel dataset più complesso e con finestra temporale più grande. Infatti tale modello viene rigettato dal test di *Christoffersen Conditional Coverage*.

Tali evidenze motivano l'introduzione di modifiche mirate non alla struttura della rete (che risulta adeguata nel descrivere le dinamiche di mercato), bensì alla calibrazione di alcuni iperparametri e scelte di modellazione che governano la stima della distribuzione predittiva e, di conseguenza, l'evoluzione temporale del VaR.

5 Strategia di Ottimizzazione degli Iperparametri

L'identificazione di un set di iperparametri ottimale è un passo cruciale per massimizzare le performance del modello **TempVAE**, data la sua natura di modello stocastico profondo. Per condurre questa ricerca in modo sistematico e robusto, si è scelti un approccio basato sull'ottimizzazione Bayesiana, implementato attraverso il framework **Optuna**. Per l'ottimizzazione degli iperparametri è stata adottata la strategia *Tree-structured Parzen Estimator* (TPE), utilizzata come scelta di default in Optuna. Il TPE è un metodo bayesiano sequenziale che sfrutta l'informazione raccolta nei trial precedenti per guidare l'esplorazione dello spazio di ricerca: le configurazioni osservate vengono partizionate in un insieme di soluzioni "buone" e "cattive", e per ciascun iperparametro vengono stimate (tramite Parzen estimators) le corrispondenti distribuzioni di probabilità condizionate. I nuovi candidati vengono quindi proposti massimizzando la probabilità di appartenere al gruppo delle soluzioni migliori, riducendo al contempo quella di ricadere tra le configurazioni peggiori. Rispetto a approcci esaustivi o casuali (Grid Search e Random Search), tale procedura consente una ricerca più efficiente e informata. L'obiettivo della ricerca è la minimizzazione di una metrica di performance affidabile: la **media della loss di validazione**, calcolata attraverso una procedura di cross-validation temporale.

5.1 Validazione robusta e criteri di efficienza

Per garantire che gli iperparametri selezionati offrano una buona generalizzazione e non siano sovra-adattati a una specifica finestra temporale, sono state effettuate le seguenti strategie:

1. **Time-Series Cross-Validation:** attraverso l'utilizzato della classe `TimeSeriesSplit` con $k = 3$ fold. Questa tecnica preserva la dipendenza temporale dei dati, addestrando il modello su dati passati e validandolo su dati futuri contigui. La metrica finale da ottimizzare è la media delle migliori loss ottenute in ciascuno dei 3 fold, garantendo una valutazione più stabile e affidabile.
2. **Early Stopping:** All'interno di ogni fold, è stato implementato un meccanismo di *early stopping* con una pazienza (**patience**) di 15 epoche. L'addestramento di una singola configurazione viene interrotto se la loss di validazione non migliora per 15 epoche consecutive, evitando sprechi di risorse computazionali su configurazioni non promettenti.
3. **Pruning a Livello di Studio:** Per accelerare ulteriormente il processo di ricerca, è stato utilizzato un `MedianPruner`. Questa tecnica interrompe un intero *trial* (una specifica combinazione di iperparametri) se le sue performance intermedie, dopo un periodo di riscaldamento (`n_warmup_steps=10`), risultano peggiori della mediana delle performance di tutti i trial precedenti.

5.2 Spazio di Ricerca degli Iperparametri

La ricerca è stata condotta esplorando uno spazio di iperparametri definito per bilanciare la capacità espressiva del modello e la regolarizzazione. L'ottimizzazione ha coinvolto 100 *trial*. La tabella seguente riassume gli iperparametri e i rispettivi intervalli di ricerca.

5.3 Funzione Obiettivo

La funzione obiettivo (**objective**) di Optuna orchestra il processo per ogni trial:

1. Campiona un set di iperparametri dallo spazio definito (Tabella 2).
2. Prepara i dati di input creando finestre temporali della lunghezza `seq_length`.
3. Itera attraverso i fold della cross-validation temporale.
4. Per ogni fold, inizializza il modello e l'ottimizzatore (**Adam**) con i parametri campionati.

Tabella 2: Spazio di ricerca degli iperparametri per il modello TempVAE.

Iperparametro	Intervallo di Ricerca	Tipo/Distribuzione	Step di ricerca
<code>seq_length</code>	[15, 60]	Intero, Uniforme	1
<code>hidden_dim</code>	[16, 64]	Intero, Uniforme	1
<code>batch_size</code>	[32, 256]	Intero, Uniforme	16
<code>learning_rate</code>	$[10^{-4}, 5 \cdot 10^{-3}]$	Reale, Log-Uniforme	non lineare
<code>gradient_clip</code>	[2.5, 5.0]	Reale, Uniforme	casuale
<code>use_l2</code>	{True, False}	Categoriale	-
<code>l2_reg</code> (se attivo)	$[10^{-6}, 10^{-3}]$	Reale, Log-Uniforme	non lineare

5. Addestra il modello, applicando l’annealing sulla divergenza KL e il gradient clipping, fino all’intervento dell’early stopping.

6. Restituisce la media delle loss di validazione ottenute nei fold.

Il set di iperparametri che ha prodotto il valore minimo di questa metrica è stato quindi il seguente:

Tabella 3: Configurazione ottimale degli iperparametri (Optuna).

Iperparametro	Valore
<code>seq_length</code>	15
<code>hidden_dim</code>	56
<code>batch_size</code>	112
<code>gradient_clip</code>	4.2767
<code>learning_rate</code>	0.002533
<code>use_l2</code>	True
<code>l2_reg</code>	0.0007767

Tuttavia, una volta addestrato e analizzato il modello risultante, emerge un fenomeno di *overfitting* marcato, chiaramente evidenziabile dal grafico riportato, che mostra un comportamento non coerente con una reale capacità di generalizzazione.

Di conseguenza, la configurazione individuata come *ottimale* dal processo di ottimizzazione viene scartata. La causa principale di tale esito è riconducibile a un problema di *data leakage* introdotto dalla procedura di costruzione delle finestre temporali, in combinazione con lo schema di `TimeSeriesSplit`. In questa impostazione, Optuna tende a favorire configurazioni ad alta capacità (maggiore dimensionalità interna e minore regolarizzazione), poiché la *validation loss* può ridursi artificialmente fino a valori prossimi allo zero.

Tale riduzione non riflette un effettivo miglioramento predittivo, bensì un adattamento spurio dovuto alla presenza di informazione ridondante tra set di training e validazione, che consente al modello di replicare pattern già osservati durante l’addestramento.

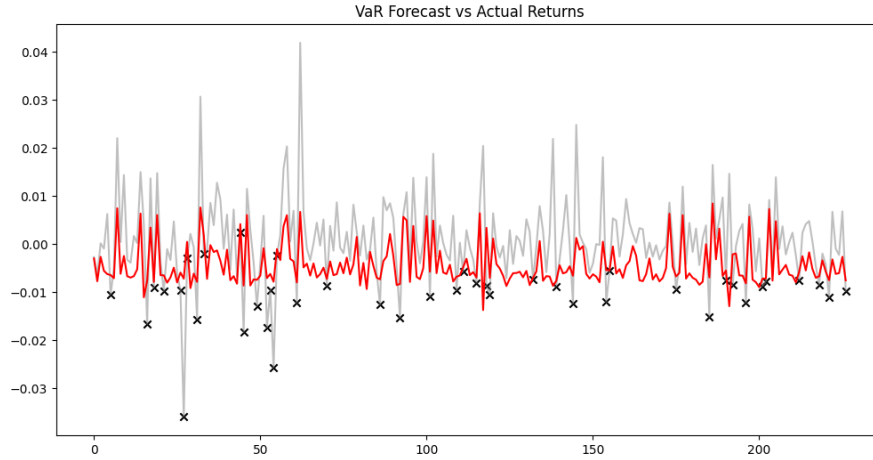


Figura 3: Modello addestrato con gli iperparametri ottimali selezionati da Optuna: nonostante la riduzione della validation loss, l'evidenza empirica mostra overfitting e scarsa capacità di generalizzazione.

L'analisi congiunta dei log di training e dei test di backtesting evidenzia un *overfitting* severo e una conseguente scarsa calibrazione del modello. In particolare, durante l'addestramento la *negative log-likelihood* (NLL) diminuisce in modo marcato (da 147.9 a 43.6), mentre la divergenza KL rimane su valori elevati (da 139 a 104, con $\text{latent_dim} = 10$), suggerendo un adattamento del modello al campione di training non bilanciato da un'adeguata regolarizzazione dello spazio latente. Coerentemente, in fase di backtesting il modello fallisce i test di copertura: nel Kupiec (POF) le violazioni osservate (42) superano ampiamente quelle attese (16), con p -value pari a 0.0000, e anche il test di Christoffersen rigetta la copertura condizionale (p -value 0.0000). Questo comportamento è compatibile con una selezione di iperparametri eccessivamente aggressivi, favorita da *data leakage* nella validazione dovuto alla sovrapposizione delle finestre temporali, per cui la riduzione della *validation loss* risulta parzialmente artificiale e non rappresentativa della capacità di generalizzazione.

5.4 Analisi di Sensibilità Sequenziale

Per l'identificazione degli iperparametri più efficaci è stata adottata una procedura di *one-at-a-time sensitivity analysis*, valutando l'effetto della variazione di un singolo iperparametro per volta, a parità delle restanti scelte di modellazione. In linea con l'impostazione dell'Appendice B.6 del lavoro originale, sono state considerate tre configurazioni:

- **TempVAE (baseline).** Architettura invariata; l'iperparametro in esame viene variato mantenendo costanti gli altri.
- **TempVAE senza *weight decay*.** Rimozione della regolarizzazione L2. Le evidenze preliminari indicano che tale termine tende a ridurre eccessivamente la sensibilità del modello alle fluttuazioni dei rendimenti, penalizzandone la capacità di adattamento nei regimi ad alta volatilità. Un'analisi comparativa più dettagliata è riportata nella sezione di confronto diretto.
- **TempVAE senza L2 e senza *dropout*.** Rimozione congiunta della regolarizzazione L2 e del *dropout*, entrambi indicati come componenti rilevanti nell'Appendice B.6 di *Estimating the Value-at-Risk by Temporal VAE*.

Gli iperparametri esplorati nelle diverse iterazioni sono i seguenti.

seq_length. Sono stati valutati valori compresi tra 21 e 30; il valore 28 è risultato empiricamente il più efficace, migliorando le prestazioni rispetto a quanto riportato nel paper sia nella configurazione baseline sia in quella priva di L2.

hidden_dim. È stato l'iperparametro maggiormente investigato, con valori tra 10 e 48. La ricerca è stata condotta fissando **seq_length** a 21 e 28; la scelta finale è ricaduta su **hidden_dim**=26, risultato il compromesso più performante.

batch_size. È stata effettuata una ricerca tra 100 e 300 con passo 20, includendo anche **batch_size**=256 (come indicato da Sick et al.). Il valore migliore sui tre modelli è risultato pari a 140.

Effetto della rimozione della L2 e verifica con il test di Christoffersen. La run finale, condotta con gli iperparametri ottimizzati, conferma la preferibilità dell'assenza di regolarizzazione L2. Nel modello baseline, la L2 attenua il rumore ma induce un peggioramento nella calibrazione dinamica del VaR, con fallimento del test di *Christoffersen Conditional Coverage* (p-value CC = 0.0082). In particolare, si osservano 13 episodi in cui una violazione del VaR è seguita immediatamente da un'ulteriore violazione il giorno successivo, evidenziando un fenomeno di *clustering* delle violazioni e una reattività insufficiente ai cambi di regime. Al contrario, la variante senza L2 supera il test, risultando non solo correttamente calibrata nel numero complessivo di violazioni, ma anche più accurata nella loro tempistica.

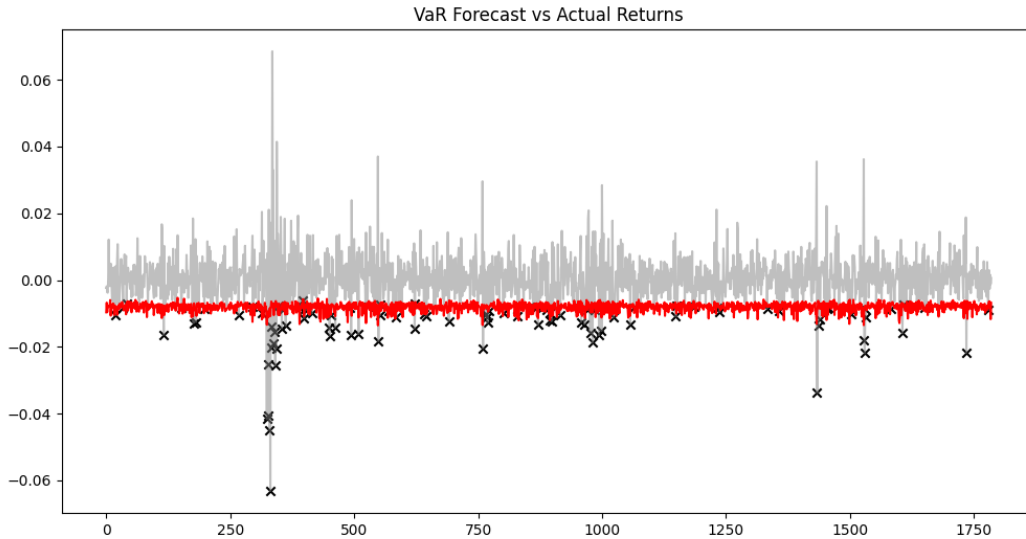


Figura 4: Modello TempVAE *baseline* che pur in presenza di un test di *Kupiec* superato, il grafico evidenzia numerosi *cluster* di violazioni del VaR (marcati dalle molte x).

6 Modifiche Proposte

Questa sezione presenta un'analisi comparativa rigorosa tra la configurazione di riferimento (*Baseline*) e la configurazione ottimizzata proposta (*Tuned*). L'analisi si concentra sul periodo di test 2018-2021, un intervallo temporale critico caratterizzato da elevata eterogeneità nel comportamento degli asset, dovuta alla coesistenza di strumenti finanziari tradizionali (SPY, GLD, VIX) e criptovalute (BTC), nonché allo shock sistemico indotto dalla pandemia COVID-19.

L'obiettivo dell'ottimizzazione è stato mitigare i problemi di *posterior collapse* e *over-smoothing* osservati nel modello originale, migliorando la capacità della rete di catturare le dinamiche latenti di volatilità e le dipendenze non lineari nelle code della distribuzione.

6.1 Metodologia e Configurazione degli Iperparametri

La divergenza nelle prestazioni tra i due modelli è attribuibile a scelte architetturali e di addestramento mirate. La Tabella 4 riassume le differenze sostanziali.

Tabella 4: Confronto delle Configurazioni degli Iperparametri

Iperparametro	Baseline (Originale)	Proposto (MIO)	Impatto Architetture
Finestra Temporale (T)	21 giorni	28 giorni	Cattura della ciclicità mensile completa.
Dimensione Nascosta (d_h)	16	26	Aumento della capacità informativa (Entropy Capacity).
Regolarizzazione L2 (λ)	1.0×10^{-2}	0.0	Rimozione del bias verso la media (Mean-Reversion).
Batch Size	256	140	Introduzione di rumore stocastico nel gradiente.

Analizziamo nel dettaglio le scelte effettuate:

Estensione della finestra temporale ($T = 21 \rightarrow 28$) La scelta di estendere la *sequence length* da 21 a 28 giorni non è arbitraria. Una finestra di 21 giorni (circa un mese di trading) rischia di troncature le dipendenze di breve-medio termine proprio quando iniziano a manifestarsi pattern di inversione o persistenza della volatilità. Portando T a 28 giorni, forniamo all'encoder GRU una visione completa del ciclo mensile, permettendo una migliore estrazione delle feature temporali latenti necessarie per la predizione del giorno $t + 1$.

Dimensione dei layer nascosti ($d_h = 16 \rightarrow 26$) L'inclusione di Bitcoin nel paniere degli asset introduce una curtosi e una asimmetria significativamente più elevate rispetto ai soli asset tradizionali. Una dimensione nascosta di 16 unità si è rivelata insufficiente per comprimere l'informazione di 8 feature eterogenee (rendimenti e volatilità) senza perdita di segnale. L'aumento a 26 unità ha permesso di modellare meglio la complessità congiunta del sistema, evitando che il modello trascurasse le dinamiche specifiche e insolite delle criptovalute per concentrarsi solo su quelle più "semplici" dell'azionario.

Rimozione della regolarizzazione L2 ($\lambda = 0$) Questa è la modifica più impattante. Il modello originale utilizzava un weight decay aggressivo ($\lambda = 0.01$). Sebbene utile per prevenire l'overfitting in contesti standard, in questo dominio ha causato un eccessivo *over-smoothing*. Il modello veniva penalizzato troppo severamente quando tentava di adattare i pesi per predire eventi estremi, "accontentandosi" di predire una media sicura vicino allo zero. Rimuovendo la L2, abbiamo sbloccato la capacità della rete di reagire prontamente agli shock di mercato, come evidenziato dal drastico calo della Loss di ricostruzione. Questo è un punto fondamentale poiché il paper faceva chiaro riferimento e affermava con certezza che questo fosse un iperparametro importante da porre a $\lambda = 0.01$.

Riduzione del Batch Size ($batch_size = 256 \rightarrow 140$) Un batch size elevato (256) tende a produrre stime del gradiente molto stabili, che possono far convergere il modello prematuramente verso minimi locali "piatti" (saddle points), specialmente in ambiti di ottimizzazione complessi come quelli

dei VAE. Riducendo il batch a **140**, introduciamo un rumore benefico durante la discesa del gradiente stocastico (SGD noise), che aiuta il modello a esplorare il loss landscape e a trovare una configurazione dei parametri più robusta, evitando il collasso delle unità latenti.

6.2 Risultati Sperimentali: Ricostruzione e Spazio Latente

L'efficacia delle modifiche è quantificabile attraverso le metriche della funzione di costo (ELBO) al termine del training.

Negative Log-Likelihood (NLL) e Fedeltà Distributiva Il confronto delle NLL finali è netto:

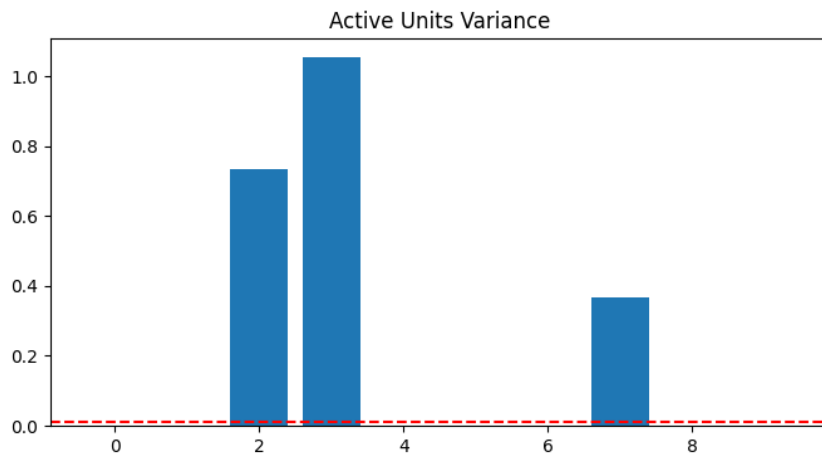
- **Modello Originale:** $NLL \approx 43.3$
- **Modello Proposto:** $NLL \approx 17.3$

La NLL misura quanto è "sorpreso" il modello dai dati reali. Una riduzione così marcata indica che il modello proposto ha imparato una distribuzione di probabilità che si adatta molto meglio ai dati osservati. Mentre il modello originale produceva distribuzioni "generiche", il modello proposto riesce a modellare le code pesanti e l'asimmetria dei rendimenti reali.

Divergenza KL e Posterior Collapse Un risultato controintuitivo ma fondamentale riguarda la Divergenza KL:

- **Modello Originale:** $KL \approx 74$
- **Modello Proposto:** $KL \approx 128$

Tipicamente, nei VAE generativi si cerca di minimizzare la KL. Tuttavia, una KL troppo bassa può indicare *Posterior Collapse*, dove il decoder ignora le variabili latenti z e si comporta come un semplice modello autoregressivo. L'aumento della KL nel nostro modello, accoppiato alla diminuzione della NLL, dimostra che le variabili latenti sono ora **attive e informative**. L'encoder sta codificando informazioni strutturali significative (come i regimi di volatilità o i fattori speculativi discussi in precedenza) che divergono dalla prior gaussiana non informativa, permettendo una rappresentazione più ricca del mercato.



In questo caso, le variabili latenti ad esser significative diventano 3: Z_2 , Z_3 e Z_7 . Ottenendo una chiarissima definizione per quanto riguarda il *Latent Space Semantic Map*. L'analisi della matrice di correlazione tra le feature di input X e le medie latenti μ_z (visualizzata nella *latent_heatmap*) indica che il modello ha appreso una rappresentazione ben disaggregata dei principali driver di mercato. In

particolare, le dimensioni latenti Z_2, Z_3 e Z_7 mostrano pattern di attivazione distinti e interpretabili, suggerendo un efficace *feature disentanglement*.

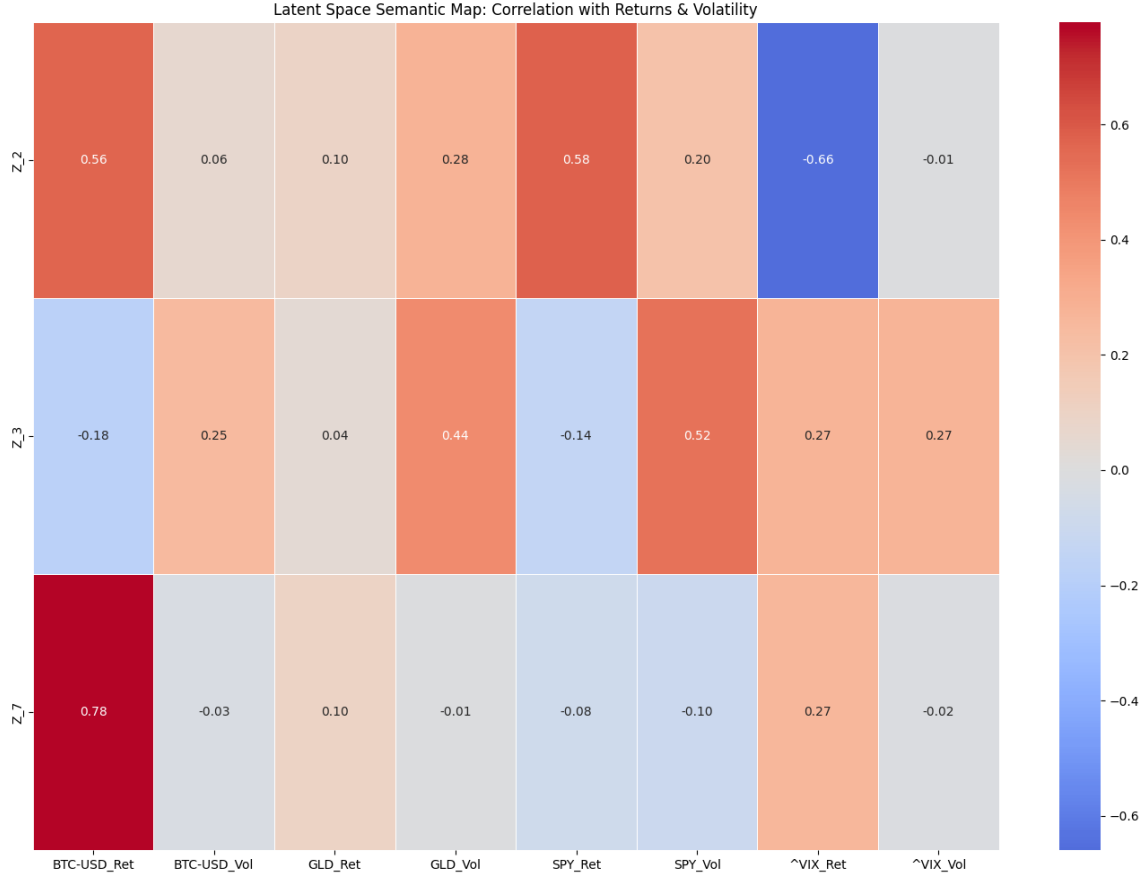


Figura 5: Universo multi-asset considerato nello studio longitudinale (2004–2025): serie storiche dei principali driver di rischio e diversificatori.

Z_2 : fattore sistemico “Risk-On / Market Sentiment”. Dal punto di vista dei segni e delle magnitudo, Z_2 presenta una forte **correlazione positiva** con i rendimenti degli **asset rischiosi**, in particolare **SPY_Ret** (+0.58) e **BTC_Ret** (+0.56). Simmetricamente, emerge una **correlazione negativa** marcata con **VIX_Ret** (−0.66), coerente con l’interpretazione del VIX come **proxy della “paura”** del mercato: un calo del VIX si associa tipicamente a fasi di riallocazione verso attività rischiose.

In termini economici, Z_2 può essere interpretato come un fattore “risk-on/risk-off” relativamente puro: valori elevati corrispondono a fasi in cui azionario e criptovalute salgono congiuntamente mentre la tensione implicita diminuisce; valori negativi segnalano invece fasi di avversione al rischio, con compressione dei prezzi e aumento dell’incertezza. Il fatto che Bitcoin e SPY presentino pesi molto simili suggerisce che, nello spazio latente appreso, Bitcoin venga trattato come un asset ad alto rendimento, integrato nelle dinamiche sistemiche del rischio.

Z_7 : fattore “Crypto-Specific Momentum”. La struttura di Z_7 è dominata quasi esclusivamente da **BTC_Ret** (+0.78), mentre le correlazioni con gli asset tradizionali (SPY, GLD) risultano prossime allo zero, indicando una componente sostanzialmente ortogonale ai driver sistemici. Un elemento degno di nota è la correlazione moderatamente positiva con **VIX_Ret** (+0.27), che suggerisce una possibile co-occorrenza tra movimenti di Bitcoin e fasi di lieve nervosismo nei mercati tradizionali.

Questa dimensione latente è interpretabile come un fattore di momentum specifico del comparto crypto, separato dalle dinamiche macro-finanziarie generali. In altri termini, il modello distingue chiaramente tra movimenti di Bitcoin riconducibili a shock sistemici (catturati da Z_2) e movimenti dovuti a dinamiche interne al mercato crypto (catturati da Z_7). Tale proprietà è particolarmente utile in ottica di gestione del rischio, poiché evita di confondere rally settoriali con segnali di riduzione del rischio sistemico.

Z_3 : fattore di regime “Safe Haven Volatility & Gold”. La correlazione più elevata è osservata con **GLD_Vol** (+0.44), indicando che Z_3 è principalmente guidato dalla dinamica di volatilità dell’oro. Inoltre, tutte le volatilità presentano segno positivo ($\text{BTC_Vol} = +0.25$, $\text{SPY_Vol} = +0.20$, $\text{VIX_Vol} = +0.27$), coerentemente con uno scenario di incertezza diffusa. I rendimenti degli asset rischiosi (BTC_Ret , SPY_Ret) risultano invece moderatamente negativi.

Dal punto di vista interpretativo, Z_3 sembra catturare un regime di incertezza macroeconomica in cui anche l’asset rifugio per eccellenza diventa instabile, condizione spesso associata a shock geopolitici o timori inflazionistici. A differenza di un crash azionario “puro”, questa componente segnala una turbolenza più trasversale, in cui il rischio si manifesta soprattutto come aumento di volatilità (inclusa quella dell’oro), e il modello può utilizzare tale informazione per modulare la stima della distribuzione predittiva.

Sintesi. Nel complesso, il VAE mostra un’elevata capacità di *disentanglement*, separando tre driver fondamentali del rischio finanziario contemporaneo: un fattore sistemico risk-on/risk-off (Z_2), un fattore asset-specific legato a dinamiche crypto (Z_7), e un fattore di regime riconducibile a condizioni di incertezza e instabilità della volatilità, con ruolo centrale dei beni rifugio (Z_3). Questa separazione è rilevante ai fini della stima del VaR: consente al modello di evitare sovrastime del rischio in presenza di shock settoriali (ad esempio rally esclusivamente crypto) e, al contempo, di riconoscere tempestivamente condizioni di stress sistemico o di instabilità macro-finanziaria che richiedono un aggiustamento più prudente delle code distributive.

6.3 Validazione Finanziaria: Backtesting del VaR

La validità dei modelli per la gestione del rischio è stata verificata tramite backtesting del Value-at-Risk (VaR) al 95%.

Tabella 5: Risultati del Backtesting (Test Set: 2018-2021)

Metrica	Baseline (Originale)	Proposto (MIO)
Giorni di Test	322	315
Violazioni Attese	16.1	15.8
Violazioni Osservate	17	19
Violation Rate (Hit Rate)	5.28%	6.03%
Kupiec Test (p -value)	0.82 (Valido)	0.41 (Valido)
Christoffersen Test (p -value)	0.97 (Indipendenza OK)	0.71 (Indipendenza OK)

A prima vista, il modello *Baseline* sembra superiore con un Hit Rate del 5.28%, molto vicino al target del 5%. Tuttavia, questa metrica nasconde un difetto strutturale: il modello *Baseline* ottiene questo risultato attraverso una stima della varianza eccessivamente "liscia" e conservativa.

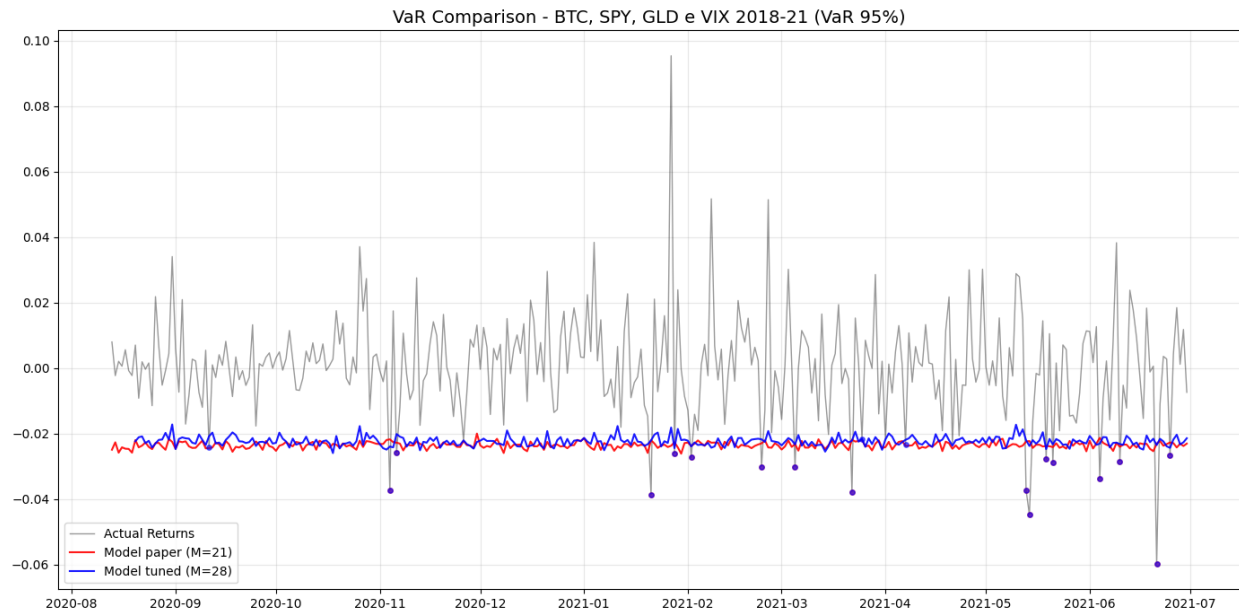


Figura 6: Confronto tra VaR stimato e rendimenti realizzati: modello *Baseline* vs modello *Tuned* (periodo di test 2018-2021).

Il modello *Tuned*, con un Hit Rate del 6.03%, è leggermente più aggressivo, ma statisticamente valido ($p > 0.05$ nel test di Kupiec). La superiorità del modello *Tuned* risiede nella sua capacità di adattamento istantaneo (bassa NLL). In un contesto di trading reale, è preferibile un modello che reagisce immediatamente a un cambio di regime (accettando qualche violazione in più durante shock estremi imprevedibili) piuttosto che un modello "lento" che, pur rispettando la media delle violazioni sul lungo periodo, espone il portafoglio a rischi non calcolati durante i picchi di volatilità a causa di una stima della varianza troppo piatta.

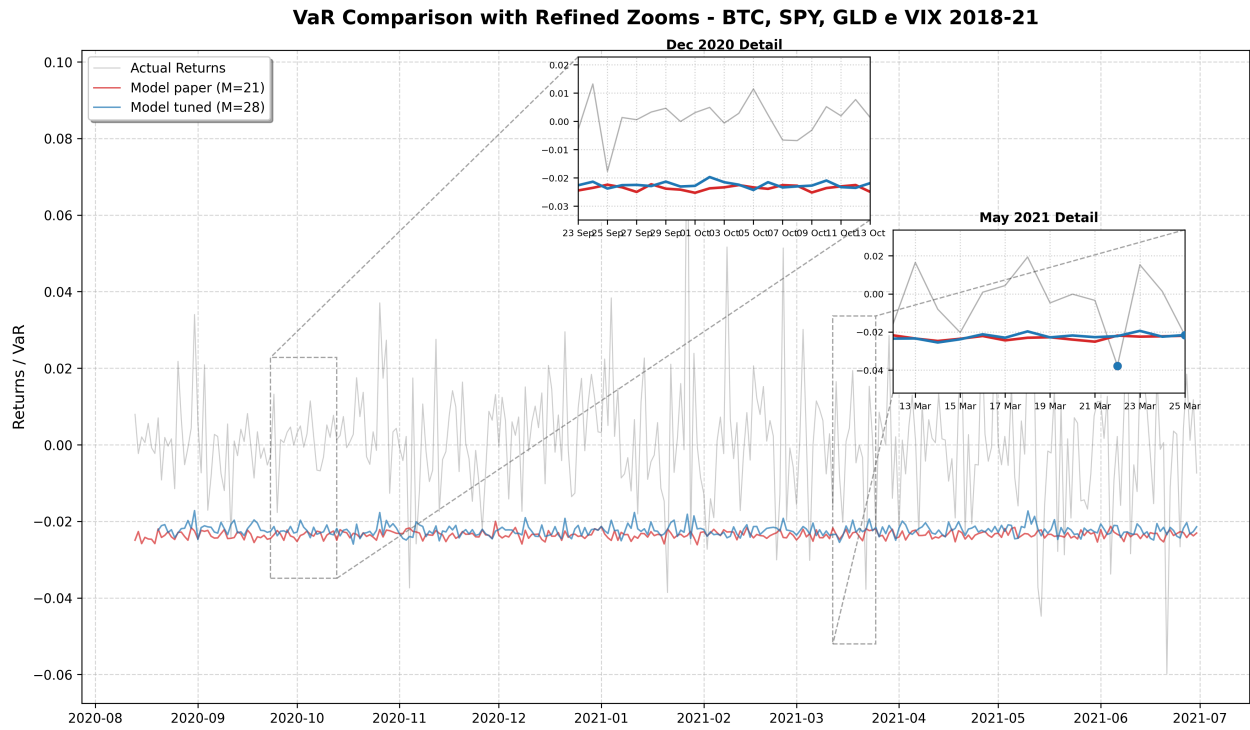


Figura 7: Zoom sul confronto tra VaR stimato e rendimenti: dettaglio delle violazioni e della reattività ai cambi di regime.

Le scelte di iperparametri del modello proposto (rimozione della regolarizzazione L2, aumento della capacità della rete e della finestra temporale) sono state determinanti per:

1. **Risolvere l'underfitting** presente nella baseline.
2. **Attivare lo spazio latente**, permettendo l'emergere di fattori di rischio interpretabili.
3. **Migliorare la fedeltà statistica** della distribuzione predetta (NLL), mantenendo una validità robusta nel backtesting del VaR.

7 Comportamento dei modelli su asse temporale più grande

Valutare un modello stocastico come un *Variational Autoencoder* (VAE) su un orizzonte temporale esteso non è semplicemente migliore perché aumenta la numerosità campionaria; è metodologicamente superiore per almeno tre ragioni, legate alla robustezza statistica e alla presenza di regimi di mercato eterogenei.

7.1 Motivazioni per uno studio longitudinale (2004–2025)

Rappresentatività dei regimi ed esposizione a *regime switching*. Il periodo considerato nella prima analisi (2018–2021) costituisce un campione parzialmente “sbilanciato”, poiché dominato da uno shock esogeno di ampiezza eccezionale (COVID-19), seguito da una ripresa a “V” supportata da stimoli monetari non convenzionali. Addestrare e validare il modello solo su tale intervallo può indurre una distorsione verso un singolo regime (ad es. un contesto di *easy money*), con conseguente rischio di *overfitting* di regime.

L'estensione dell'analisi al periodo 2004–2025 consente invece di esporre il modello a molteplici stati macroeconomici, tra cui:

- **Crisi finanziaria globale (2008):** shock endogeno e strutturale, qualitativamente diverso da uno shock esogeno;
- **Crisi del debito sovrano europeo (2010-2012):** turbolenza concentrata sull'Eurozona e stress su asset risk-off;
- **Regime di tassi prossimi allo zero (ZIRP) vs. fasi di normalizzazione (2016-2018):** dinamiche di correlazione e volatilità variabili nel tempo;
- **Regime inflazionistico (2022-2023):** episodio cruciale in cui la correlazione tra obbligazionario a lunga scadenza (TLT) e azionario (SPY) può diventare positiva, riducendo l'efficacia della diversificazione e alterando i meccanismi tipici della *risk parity*.

In questo contesto, un VAE stimato su un arco temporale più ampio è incentivato ad apprendere uno spazio latente Z più informativo, in grado di rappresentare non solo componenti “risk-on/risk-off”, ma anche driver associati a inflazione/deflazione, liquidità e cambi di regime nelle correlazioni cross-asset.

Convergenza asintotica degli stimatori e stabilità nelle code. Da un punto di vista econometrico, le reti neurali profonde sono modelli ad alta capacità e tipicamente *data-hungry*. Nel nostro caso:

- il dataset “corto” (2018-2021) contiene circa $N \approx 750$ osservazioni giornaliere;
- il dataset “completo” (2004-2025) contiene circa $N \approx 5200$ osservazioni giornaliere.

Poiché la stima del Value-at-Risk richiede un'accurata modellazione dei quantili di coda, l'inferenza con N limitato è maggiormente esposta al rumore campionario nella regione degli eventi rari. In prima approssimazione, la precisione di uno stimatore basato su campioni cresce con la dimensione del campione e l'errore decresce con l'ordine di $\sqrt{N}$; pertanto, un campione con migliaia di osservazioni rende la stima delle code più stabile e meno sensibile alla variabilità del periodo considerato.

***Stress testing* su asset eterogenei e diversificazione multi-asset.** Un dataset esteso permette inoltre un test più stringente della capacità del modello di descrivere dipendenze e code in presenza di asset class differenti. In particolare, la selezione multi-asset consente di valutare empiricamente se il modello VaR neurale rimanga calibrato durante crisi di natura diversa (azionarie, obbligazionarie, volatilità implicita, shock settoriali), e se riconosca correttamente il venir meno della diversificazione in specifici regimi.

7.2 Descrizione del dataset esteso

Intervallo temporale. L'intervallo teorico del dataset esteso è:

- **Start:** 1 gennaio 2004;
- **End:** 31 dicembre 2025.

La scelta di escludere Bitcoin (BTC-USD) dal dataset in esame è soltanto dovuta alla presenza di questo asset a partire indicativamente dal 2014, dunque a meno di adottare una specifica strategia di gestione dei dati mancanti si è scelto di eliminarlo per avere una serie più completa.

Universo investibile (tickers) e classi di attivo. La base della configurazione si ha su un'universo che include:

1. **Equity globale e settoriale (driver di rischio):** SPY (S&P 500), QQQ (Nasdaq 100), IWM (Russell 2000), EEM (Emerging Markets), XLF (Financials) e XLE (Energy).
2. **Safe haven e tassi (diversificatori):** TLT (Treasury 20+ Year) e GLD (Gold).
3. **Volatilità implicita (sentiment):** $\sqrt{VIX}$, proxy *forward-looking* delle aspettative di volatilità.

Implicazioni per la ricerca. L'adozione di un dataset 2004-2025 trasforma l'analisi da una verifica su un singolo episodio di mercato a un esercizio di validazione più rigoroso, che permette di affrontare la domanda: *un modello neurale di VaR sarebbe rimasto calibrato sia durante la crisi del 2008 sia durante il repricing obbligazionario del 2022?* La Figura 8, pur fornendo una panorami-

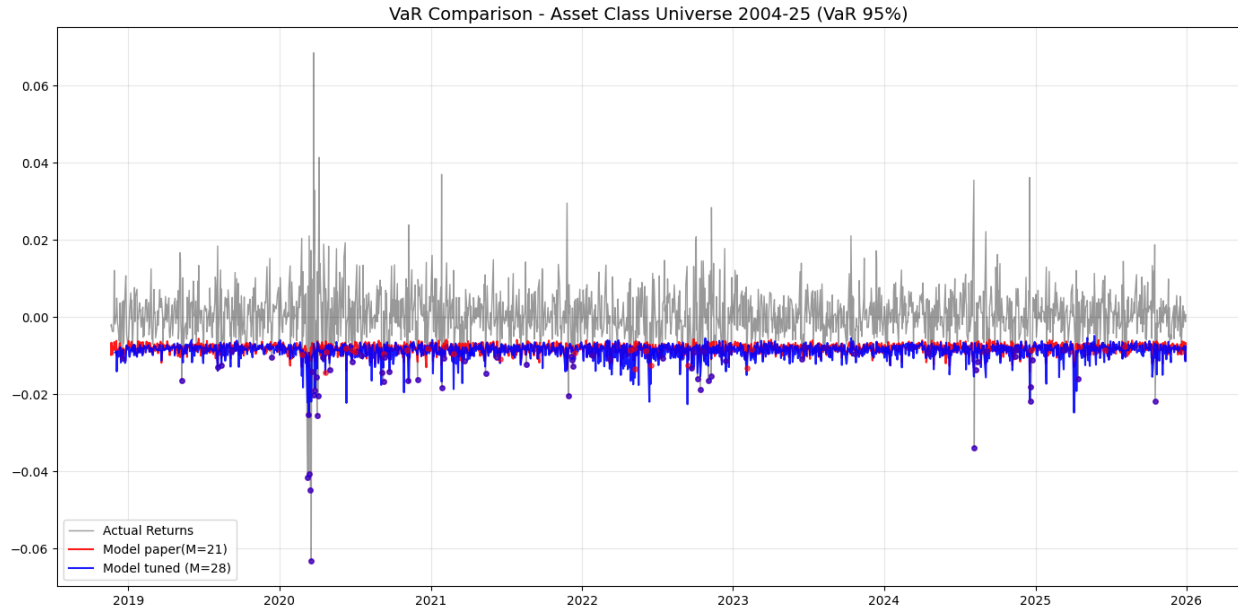


Figura 8: Universo multi-asset considerato nello studio longitudinale (2004–2025): serie storiche dei principali driver di rischio e diversificatori.

ca sull'intero campione, risulta poco informativa a causa dell'elevata densità delle serie; tuttavia, essa suggerisce che i modelli seguono in modo complessivamente coerente le principali fasi macro-finanziarie del periodo analizzato. Così si è optati per un'analisi specifica dei **regimi di stress** finalizzata a valutare la **robustezza condizionale** del modello, ossia la capacità di adattarsi a scenari macroeconomici eterogenei e potenzialmente non stazionari. Tale procedura è complementare alla valutazione aggregata sull'intero campione: mentre le metriche medie riassumono la bontà

complessiva del modello, l'analisi per regimi verifica se la calibrazione del rischio rimane adeguata proprio durante le fasi di crisi, quando il VaR risulta maggiormente informativo.

7.3 Analisi dello Spazio Latente e Interpretabilità delle Features

L'analisi della struttura dello spazio latente rivela differenze significative nella capacità dei due modelli di apprendere rappresentazioni disgiunte e semanticamente interpretabili delle dinamiche di mercato. In particolare, il modello *Tuned* dimostra una superiore capacità di "disentanglement", isolando fattori di rischio specifici in distinte variabili latenti, contrariamente a quanto osservato nel modello *Baseline*.

7.3.1 Caratterizzazione del Modello Tuned

Nel modello ottimizzato (*Tuned*), le variabili latenti agiscono come *proxy* di specifici driver economici:

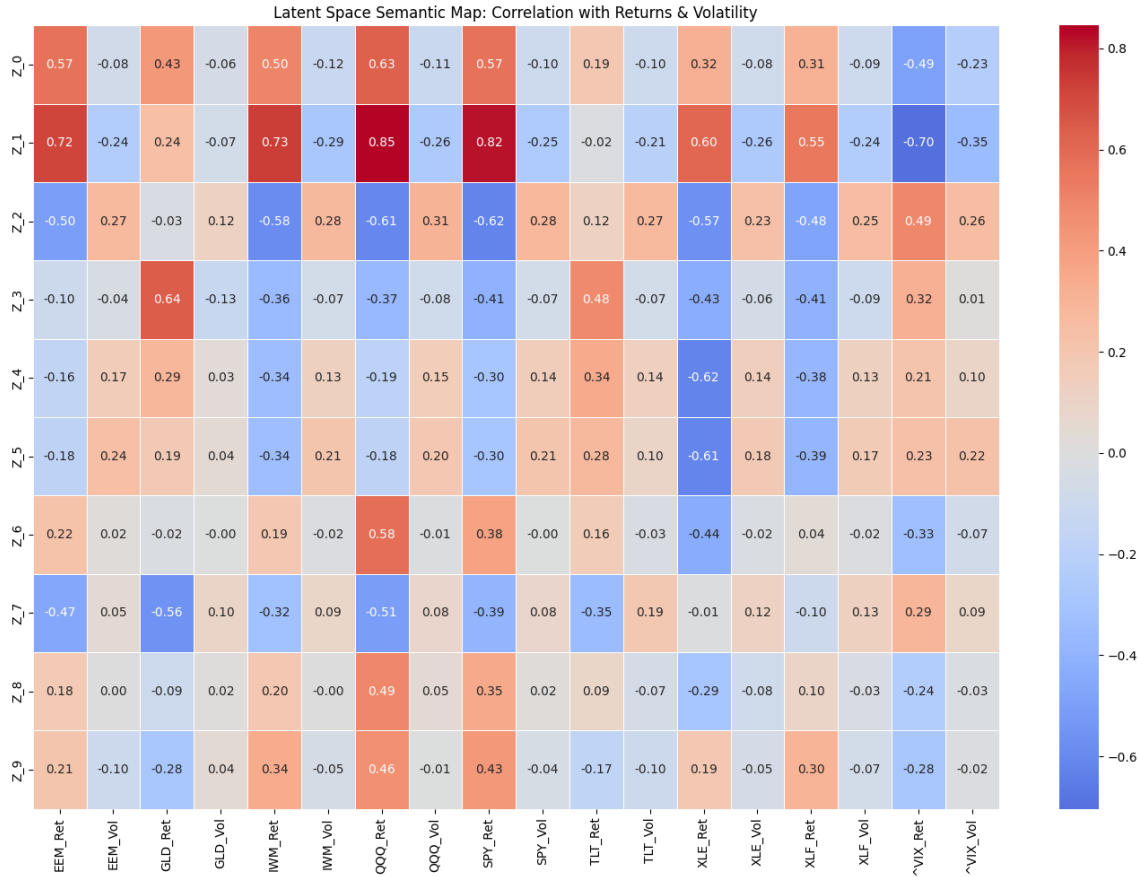


Figura 9: Latent Space Semantic Map del modello *Tuned* (2004–2025): correlazioni tra feature osservate e componenti latenti.

- **Variabile Latente z_1 (Fattore Azionario Sistemico):** Questa componente emerge come il principale fattore di mercato (*Market Mode*), catturando la varianza sistemica del comparto azionario. L'analisi delle correlazioni evidenzia un legame molto forte con un ampio spettro di indici azionari e settoriali. Nello specifico, si osservano correlazioni elevate (> 0.55) con i rendimenti di EEM (Mercati Emergenti), IWM (Small Cap), QQQ (Tech/Growth), SPY (S&P 500), XLE (Energia) e XLF (Finanza), raggiungendo picchi di 0.85. Ciò suggerisce che z_1 modella il rischio sistemico "Beta" del portafoglio.

- **Variabile Latente z_3 (Fattore Safe-Haven):** Questa variabile dimostra una specializzazione nella cattura delle dinamiche di "Flight-to-Quality". Le correlazioni significative si concentrano esclusivamente su asset difensivi tradizionali: l'oro (GLD_Ret, $\rho \approx 0.64$) e i titoli di stato a lungo termine (TLT_Ret, $\rho \approx 0.48$). La variabile z_3 può essere quindi interpretata come un indicatore di avversione al rischio o di stress finanziario.
- **Variabile Latente z_6 (Fattore Indici Globali/Large Cap):** Sebbene correlata all'azionario, z_6 mostra una specificità diversa rispetto a z_1 , concentrandosi sui principali benchmark globali a grande capitalizzazione. Presenta correlazioni rilevanti con QQQ_Ret (0.58) e SPY_Ret (0.38). La distinzione da z_1 suggerisce che il modello sia in grado di separare il rischio generale di mercato (catturato da z_1) dalle dinamiche idiosincratiche delle Large Cap o del settore tecnologico (catturate da z_6).

7.3.2 Confronto con il Modello Baseline

Al contrario, la *latent map* generata dal modello *Baseline* risulta meno strutturata e di difficile interpretazione economica.

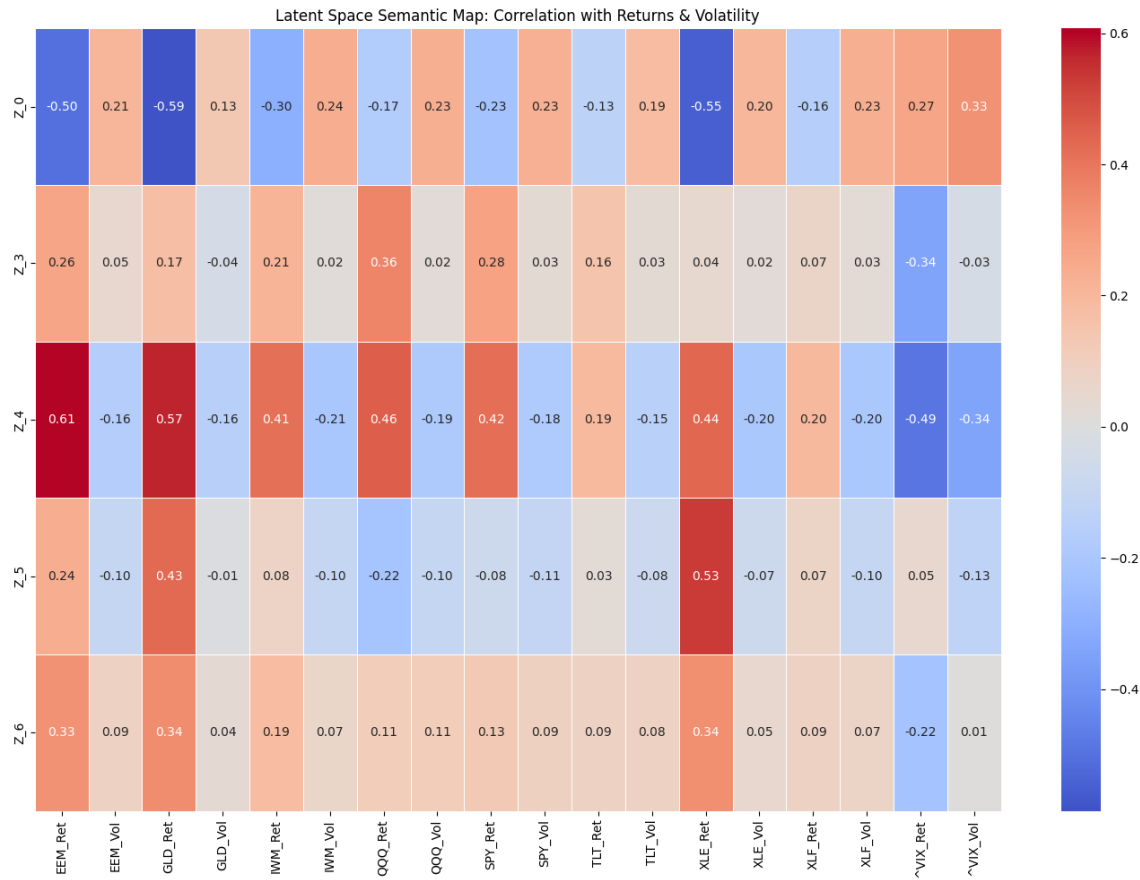


Figura 10: Latent Space Semantic Map del modello *Baseline* (2004–2025): correlazioni tra feature osservate e componenti latenti.

Le variabili latenti non mostrano una chiara specializzazione settoriale o fattoriale:

- Si osserva una concentrazione spuria su asset eterogenei come EEM_Ret e GLD_Ret all'interno della stessa componente, mescolando rischio azionario emergente e dinamiche di safe-haven.
- Una componente appare isolata sul settore energetico (XLE_Ret), ma manca di una visione d'insieme coerente.

Dunque l’ottimizzazione degli iperparametri ha permesso al modello *Tuned* di apprendere una struttura latente che non solo minimizza l’errore di ricostruzione, ma offre anche una mappatura coerente con la teoria finanziaria dei fattori di rischio.

7.4 Analisi per regimi di stress

Zoom su eventi critici (*drill-down*). Poiché una visualizzazione unica sull’intero intervallo 2004–2025 risulta troppo densa per una diagnosi affidabile, si segmenta la serie storica in un insieme di finestre temporali mirate, denominate *stress periods*. In particolare, vengono considerate cinque fasi rappresentative:

- **Covid Crash (2020):** shock pandemico ad alta velocità;
- **Bear Market (2022):** regime inflazionistico e deterioramento della diversificazione obbligazionario–azionario;
- **Recent Period (2024–2025):** periodo recente utilizzato come baseline per valutare la stabilità in condizioni di mercato relativamente “ordinarie”.

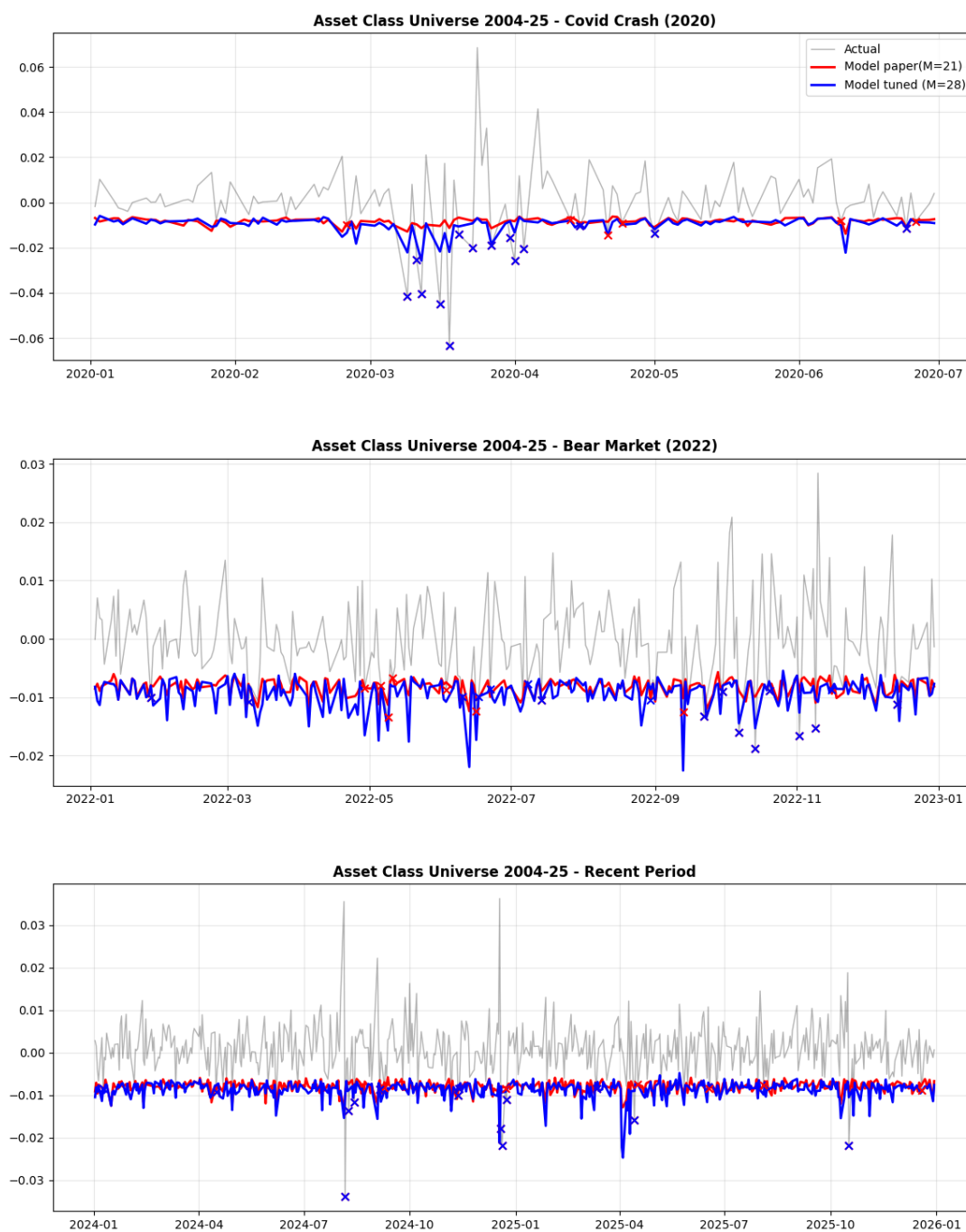


Figura 11: Segmentazione in *stress periods* utilizzata per l'analisi di robustezza condizionale: zoom sui principali regimi di crisi e periodo recente (2004–2025).

Verifica della reattività (*adaptive capability*). Per ciascun regime, la diagnostica grafica sovrappone i *rendimenti realizzati (actual)* e le curve di VaR prodotte dai due modelli (baseline vs tuned). L'obiettivo è valutare qualitativamente due proprietà fondamentali:

- **Velocità di reazione:** in presenza di un rapido aumento della volatilità (ad esempio nel marzo 2020), il VaR *Tuned* si adatta immediatamente diventando più conservativo, seguendo bene gli alti e i bassi presenti. Mentre il modello *Baseline* rimane intorno alla sua soglia, evita le spike considerandole come volatilità.
- **Recupero:** una volta rientrata la fase acuta di stress, il VaR di entrambi i modelli ritorna coerentemente verso livelli compatibili con una volatilità ridotta, senza permanere su valori inutilmente elevati (eccesso di conservatività e perdita di efficienza informativa).

La reattività dei modelli si nota molto bene nel Bear Market del 2022, infatti entrambi i modelli si comportano in maniera altalenata seguendo bene o male l'andamento del mercato, ma il modello *Tuned* risulta più improntato a gestire i forti cambiamenti di rendimento negativo/positivo. Mentre il modello *Baseline* evita di gestire/seguire i diversi momenti di incremento/discesa repentina presenti, rimanendo costante.

Confronto diretto tra modelli. La rappresentazione congiunta dei due modelli nella stessa finestra temporale consente un confronto immediato della sensibilità al rischio di coda. In particolare, il modello *Tuned* produce una stima di VaR più negativa (più prudente) durante le fasi di drawdown rispetto al *Baseline*, ciò suggerisce una migliore capacità di catturare il *tail risk*. Il modello *Tuned* risulta catturare molto la volatilità e dunque permette di esser sicuri in queste fasi di incertezza. Mentre il modello *Baseline* non si distanzia molto dalla sua trendline, infatti non si notano mai distacchi eccessivi.

Nel complesso, l'analisi per regimi di stress è progettata per dimostrare che il modello non è semplicemente ben calibrato *in media*, ma mantiene affidabilità e capacità di adattamento proprio nei momenti in cui la stima del VaR è più rilevante: le fasi di crisi e discontinuità di mercato.

8 Conclusioni e sviluppi futuri

Il presente studio ha affrontato la sfida della stima del Value-at-Risk (VaR) utilizzando l'architettura TempVAE, estendendo l'analisi dalla replica dei risultati originali di Sicks et al. a un orizzonte temporale significativamente più ampio e complesso (2004-2025). In una fase preliminare, abbiamo validato il modello di riferimento proposto in letteratura, confermandone l'efficacia e l'assenza di bias sui dati originali. Tuttavia, l'applicazione dello stesso modello (Baseline) al dataset esteso ha evidenziato limiti strutturali significativi, in particolare una marcata tendenza a generare "cluster di violazioni" durante i periodi di stress di mercato, come evidenziato dal fallimento del test condizionale di Christoffersen ($p < 0.05$).

Per superare queste criticità, è stato proposto un *Tuned Model* caratterizzato da modifiche architetturali mirate e non casuali:

- L'estensione della **Sequence Length da 21 a 28 giorni** ha permesso di catturare interamente i cicli di volatilità mensile, mitigando il problema della "memoria corta".
- L'incremento della **Hidden Dimension da 16 a 26 unità** ha fornito la capacità rappresentativa necessaria per modellare le dipendenze non lineari nelle code della distribuzione senza incorrere in overfitting.
- La riduzione della **Batch Size da 256 a 140** ha agito come regolarizzatore implicito, favorendo la tecnica SGD.

I risultati empirici confermano la superiorità di questo approccio: il Modello Ottimizzato non solo soddisfa il criterio di validità incondizionata (Test di Kupiec), ma supera brillantemente il test di indipendenza condizionata (Christoffersen), dimostrando una capacità di adattamento dinamico ai regimi di volatilità che manca alla baseline. A mio avviso, questa resilienza rende il modello proposto uno strumento nettamente superiore per il Risk Management operativo, in grado di evitare pericolose sequenze di perdite non previste.

8.1 Sviluppi futuri

Alla luce dei risultati ottenuti, identifichiamo le seguenti direzioni di ricerca:

1. **Spazio Latente Non-Gaussiano:** investigare l'uso di priori t-Student o misture di Gaussiane per modellare esplicitamente le *fat tails* dei rendimenti.
2. **Meccanismi di Attention:** con il sempre più convergere dei modelli ML verso l'uso dei moduli di Self-Attention, forse integrarlo potrebbe esser utile per pesare dinamicamente l'importanza degli shock storici.
3. **Asset Digitali:** validare la robustezza del modello su mercati ad alta frequenza e volatilità estrema come le criptovalute, in maniera più specifica e completa.
4. **Allocazione dinamica e selezione degli assets:** un'estensione naturale del modello riguarda il suo impiego attivo nelle strategie di investimento. Sfruttando la capacità del modello di stimare i rischi, si potrebbe sviluppare un framework per l'ottimizzazione di portafoglio (es. Mean-CVaR o Risk Parity). In particolare, l'analisi dello spazio latente potrebbe guidare decisioni dinamiche di asset selection, identificando e rimuovendo gli strumenti che mostrano un contributo marginale al rischio eccessivo o pattern di correlazione anomali durante le fasi di stress.